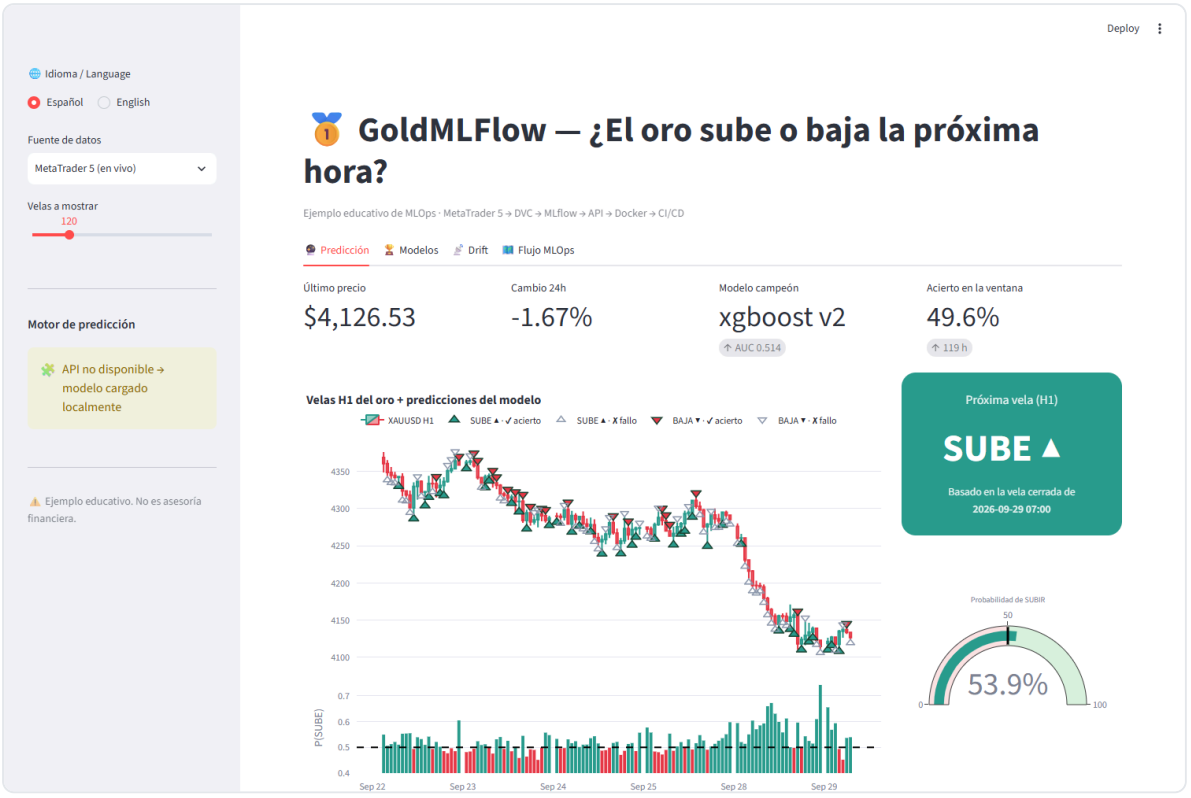
GOLDMLFLOW

MLOps para Dummies

Predecir si el oro SUBE o BAJA en la próxima hora — todo el flujo MLOps, paso a paso

MetaTrader 5 → DVC → MLflow → FastAPI → Docker → GitHub Actions → Streamlit



Código: github.com/MirandaCR/GoldMLFlow · 2026-09-28

Proyecto educativo · No es asesoría financiera

📚 Contenido

00 · La foto completa — ¿Qué es MLOps?

01 · Instalación y ejecutar todo

02 · Extraer datos del oro desde MetaTrader 5

03 · Control de versiones de datos (DVC)

04 · Validar datos y construir features

05 · Entrenar modelos y registrarlos con MLflow

06 · Evaluar, control de calidad y Registro de Modelos

07 · API del modelo (FastAPI)

08 · Contenedores (Docker)

09 · CI/CD con GitHub Actions

10 · Monitoreo y data drift

11 · Dashboard en Streamlit: ver y usar el modelo

12 · Del notebook al código de producción

13 · Glosario y preguntas frecuentes

00 · La foto completa — ¿Qué es MLOps?

🤔 El problema que resuelve MLOps

Entrenar un modelo en un notebook es como **cocinar un plato espectacular en casa**. MLOps es **montar un restaurante** que sirve ese plato todos los días, con el mismo sabor, de forma segura y a tiempo, y que cambia la receta cuando cambian los gustos de los clientes.

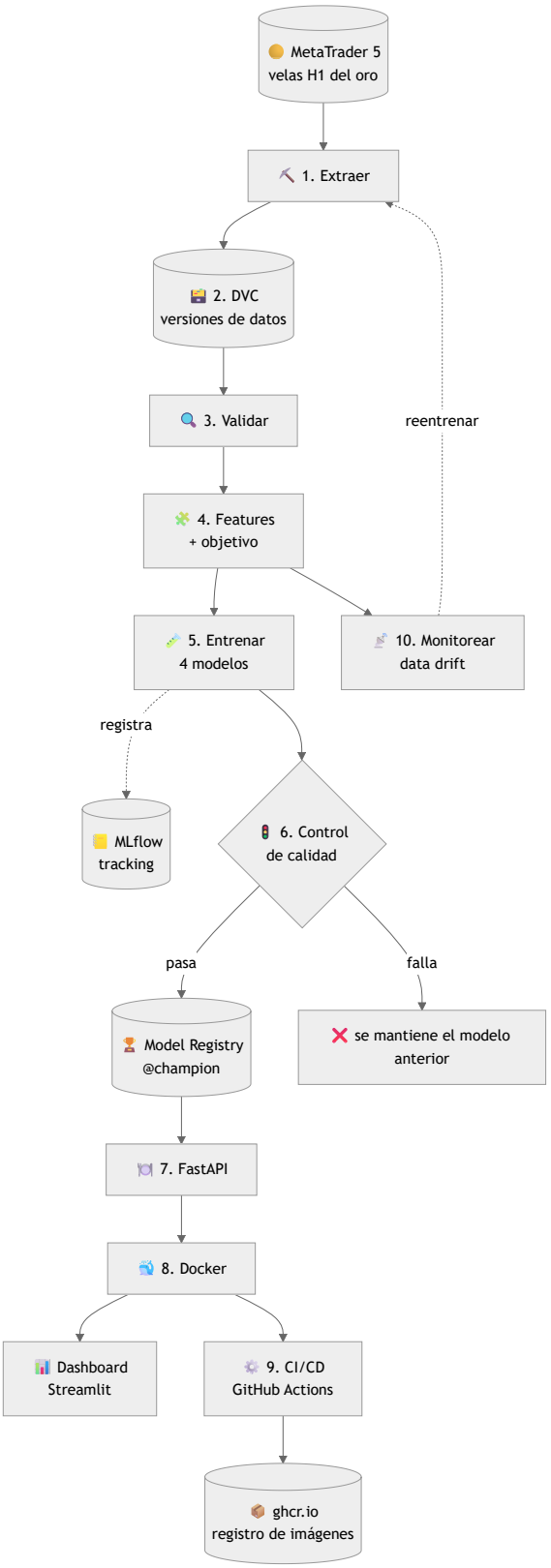
En casa (solo un notebook) 🏠	Restaurante (MLOps) 🍽️
"¿De dónde saqué esos datos?"	Los datos están **versionados** (DVC): puedes volver a cualquier versión
"¿Con qué configuración saqué 0,51?"	Cada experimento queda **registrado** (MLflow)
"En mi laptop funciona"	Está **empaquetado en un contenedor** (Docker) y corre en cualquier lado
"Pasé el modelo por correo"	El modelo se sirve mediante una **API**
"Ojalá no haya roto nada"	**Pruebas automáticas** en cada cambio (CI)
"Despliego a mano los viernes"	**Entrega automática** cuando las pruebas pasan (CD)
"El modelo empeoró… ¿desde cuándo?"	El **monitoreo** avisa cuando el mercado cambia (drift)

🥇 Nuestro ejemplo en una frase

> **Cada hora, predecir si la siguiente vela H1 del oro (XAUUSD) cerrará ARRIBA (1) o ABAJO (0)**, usando datos de MetaTrader 5, y hacer todo lo que la rodea al estilo MLOps.

Es un problema de **clasificación binaria** sobre una **serie de tiempo**.

🗺️ El flujo completo



🍳 La analogía del restaurante (tenla presente en cada capítulo)

Etapa	Herramienta	Analogía
Extraer	MetaTrader 5 + Python	🛒 Comprar los ingredientes en el mercado
Versionar datos	**DVC**	📸 Fotos con fecha de tu despensa: siempre puedes volver atrás
Validar	chequeos con pandas	🔍 Revisar que los ingredientes no estén podridos
Features	pandas	🔪 Picar y preparar los ingredientes
Entrenar	scikit-learn, XGBoost	👩‍🍳 Probar varias recetas
Registrar	**MLflow**	📒 Un cuaderno de laboratorio donde se anota cada intento
Control + registro	**MLflow Model Registry**	🏅 El jurado elige el plato que entra al menú
API	**FastAPI**	🧑‍🍳 El mesero que toma pedidos y trae el plato
Contenedor	**Docker**	🚚 Un food-truck: toda la cocina lista para estacionarse en cualquier lugar
CI/CD	**GitHub Actions**	👮 El inspector de sanidad revisa cada cambio antes de abrir
Dashboard	**Streamlit**	🪑 El comedor donde los clientes ven y disfrutan la comida
Monitorear	prueba KS de drift	📋 Revisar si los clientes siguen pidiendo lo mismo

📁 ¿Dónde está cada pieza?

```
GoldMLFlow/
├── params.yaml              ← 🎛️ panel de control: toda la configuración vive aquí
├── dvc.yaml / dvc.lock      ← 🗂️ receta del pipeline + foto exacta de cada etapa
├── src/
│   ├── data/extract.py      ← ⛏️ etapa 1
│   ├── data/validate.py     ← 🔍 etapa 3
│   ├── features/build.py    ← 🧩 etapa 4
│   ├── models/train.py      ← 🧪 etapa 5
│   ├── models/promote.py    ← 🚦🏆 etapa 6
│   ├── api/main.py          ← 🍽️ etapa 7
│   └── monitoring/drift.py  ← 📡 etapa 10
├── app/dashboard.py         ← 📊 app de Streamlit
├── notebooks/               ← 📓 exploración (uno por etapa) → se convirtieron en src/*.py
├── tests/                   ← ✅ pruebas automáticas (las ejecuta el CI)
├── Dockerfile(.dashboard), docker-compose.yml  ← 🐳
└── .github/workflows/       ← ⚙️ CI (ci.yml) y CD (cd.yml)
```

⚠️ Expectativas honestas

Los movimientos horarios del oro son **casi aleatorios**. Nuestro mejor modelo acierta **~51,5 %** de las veces (el baseline "siempre SUBE" acierta ~50,9 %). Es normal y **no** es el objetivo: este proyecto enseña el **proceso**. Si algún modelo de trading muestra 90 % de acierto, sospecha de un error (ver fuga de datos).

> 📌 Proyecto educativo. **No es asesoría financiera.**

---

01 · Instalación y ejecutar todo

🍳 **Analogía**	Preparar la cocina antes de cocinar
🎯 **Objetivo**	Tener listos Python, las librerías, MT5, Docker y Git
⏱️ **Tiempo**	~15 minutos

✅ Requisitos

Herramienta	Para qué	Verificar
Windows 10/11	El paquete `MetaTrader5` de Python solo existe para Windows	—
Terminal MetaTrader 5 (abierta y con sesión iniciada; sirve una cuenta demo)	Fuente de los precios del oro	abrir la app
Python 3.11 o 3.12	Ejecuta todo	`python --version`
Git	Versiona el código	`git --version`
Docker Desktop	Contenedores	`docker --version`
Cuenta de GitHub	CI/CD	—

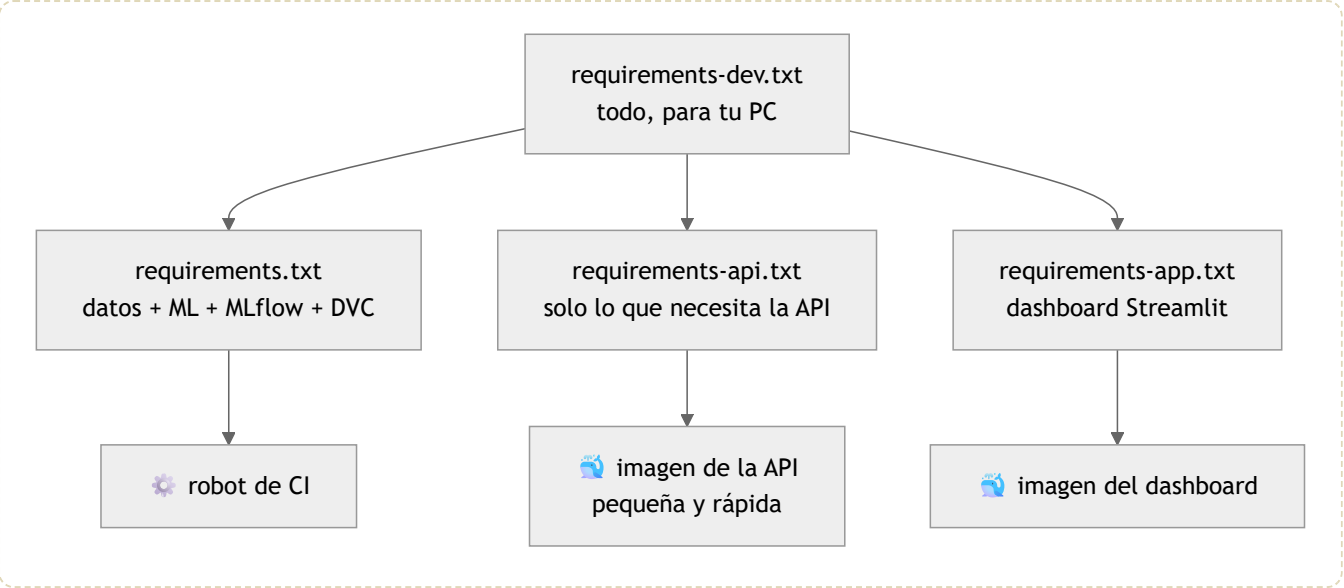
🧰 Paso a paso

```
# 1) Descargar el código
git clone https://github.com/MirandaCR/GoldMLFlow.git
cd GoldMLFlow

# 2) Crear un entorno de Python aislado ("una cocina limpia solo para esta receta")
python -m venv .venv
.\.venv\Scripts\Activate.ps1

# 3) Instalar todo (núcleo + API + dashboard + notebooks + pruebas)
pip install -r requirements-dev.txt
```

📦 ¿Por qué 4 archivos de requirements?



Cada contenedor instala **solo lo que necesita** → imágenes más pequeñas, rápidas y seguras.

🚀 Ejecutar todo el pipeline (un solo comando)

Con MetaTrader 5 **abierto**:

```
dvc repro          # extraer → validar → features → entrenar → promover → drift
```

Verás cada etapa correr en orden. Ejecútalo otra vez y DVC dirá "*didn't change, skipping*": solo re-ejecuta lo que cambió.

👀 Ver los resultados

```
mlflow ui --backend-store-uri sqlite:///mlflow.db     # 🧪 experimentos → http://localhost:5000
uvicorn src.api.main:app                               # 🍽️ API          →
http://localhost:8000/docs
streamlit run app/dashboard.py                         # 📊 dashboard    → http://localhost:8501
docker compose up --build                              # 🐬 API + dashboard en contenedores
python scripts/live_demo.py                            # 🔮 predicción en vivo de MT5 en la
terminal
jupyter lab notebooks/                                 # 📓 los notebooks de exploración
pytest                                                 # ✅ pruebas
```

> 💡 **¿El puerto 8000 está ocupado?** (otra app lo usa) → `$env:API_PORT=8010; docker compose up` y usa `http://localhost:8010`.

🧪 ¿No tienes MetaTrader 5? (Mac/Linux o sin cuenta)

Todo, salvo la extracción, funciona con la **muestra** guardada en Git (`data/sample/xauusd_h1_sample.csv`, últimas 4.000 velas):

```
python -m src.data.validate  --input data/sample/xauusd_h1_sample.csv
python -m src.features.build --input data/sample/xauusd_h1_sample.csv --output
data/processed/features.csv
python -m src.models.train
python -m src.models.promote
```

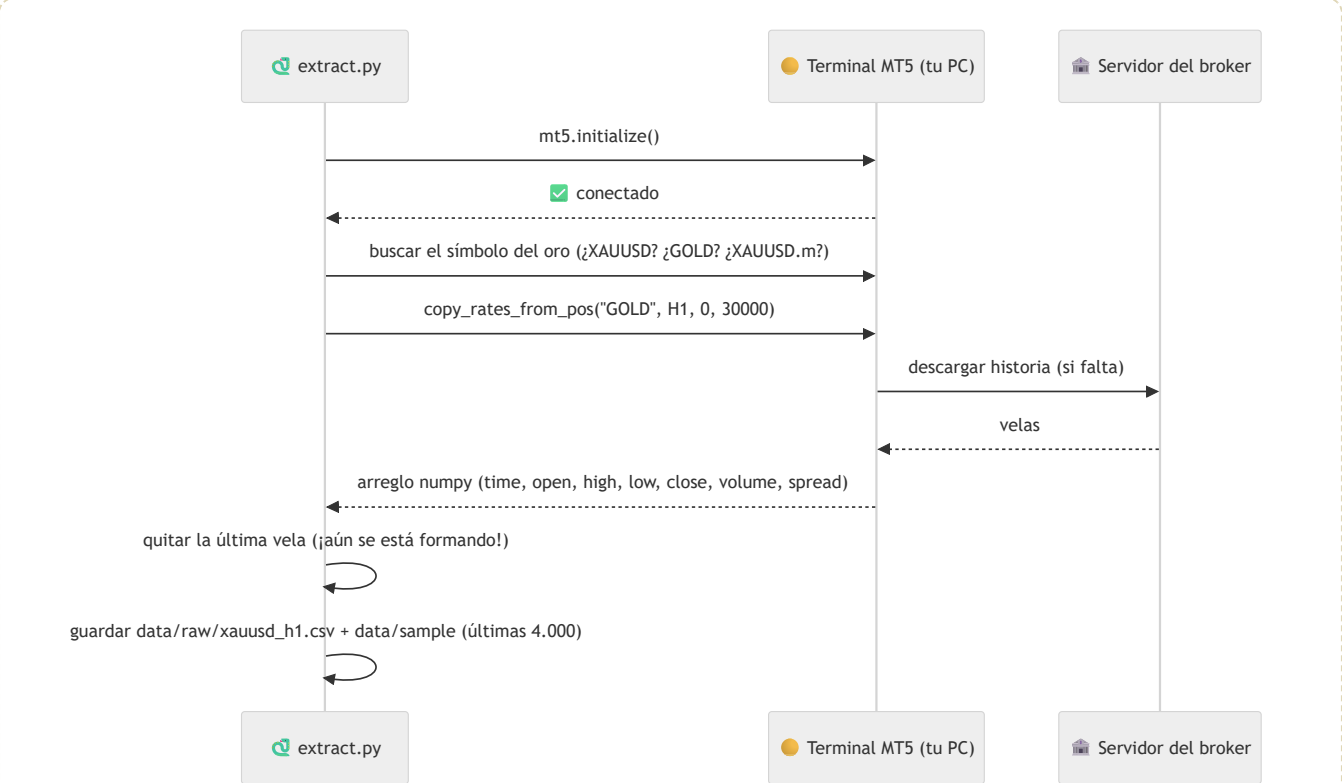
Eso es exactamente lo que hace el robot de CI en GitHub (Linux, sin MT5).

---

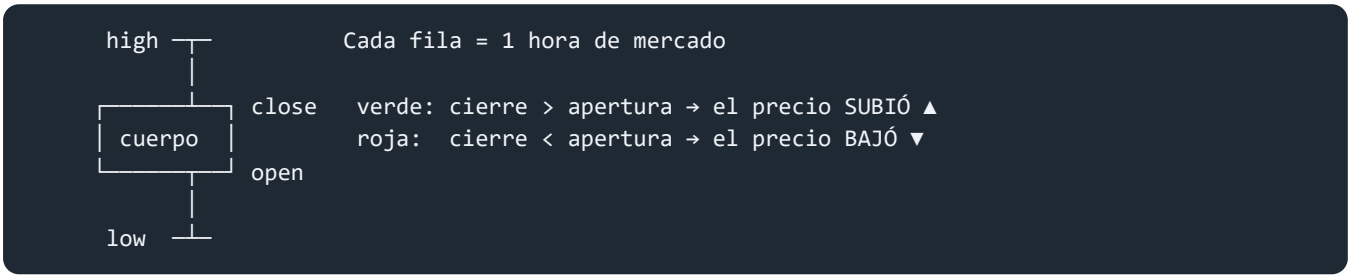
02 · Extraer datos del oro desde MetaTrader 5

🍳 **Analogía**	🛒 Ir al mercado a comprar los ingredientes
🎯 **Objetivo**	Descargar velas horarias (H1) del oro a un CSV
📓 **Notebook**	`notebooks/01_extract_mt5.ipynb`
🏭 **Código de producción**	`src/data/extract.py`
▶️ **Ejecutar**	`dvc repro extract` (o `python -m src.data.extract --sample`)

🔌 Cómo funciona



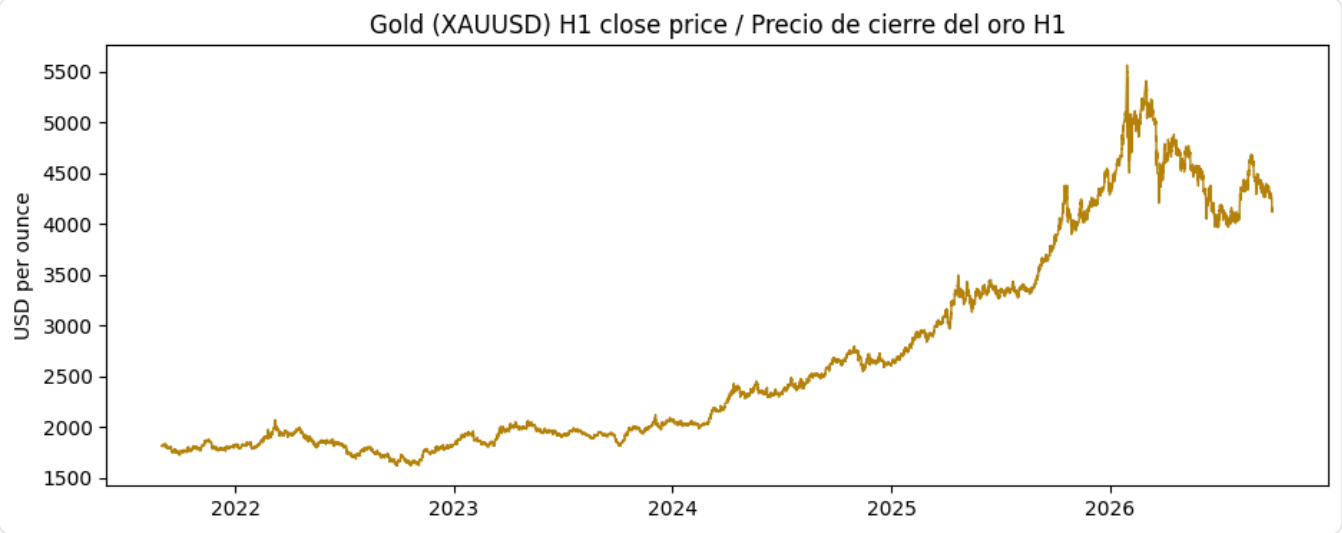
🕯️ ¿Qué es una vela?



time	open	high	low	close	volume
2021-08-31 18:00	1807.03	1808.94	1804.15	1807.78	9978
2021-08-31 19:00	1807.86	1814.04	1807.42	1813.83	5287

📊 ¿Cuánta historia hay?

Nuestro broker llama al oro `GOLD` y tiene **82.525 velas H1 (junio 2001 → hoy)**. Usamos las **últimas 30.000 (~5 años: ago 2021 → sep 2026)**.



¿Por qué no los 25 años? Los mercados cambian: el oro en 2005 tenía otra volatilidad, liquidez y horarios. Cinco años dan ~30 mil ejemplos y siguen siendo relevantes. Es un parámetro (`params.yaml → data.n_bars`), así que puedes probar con más.

🧠 Detalles que importan en producción

Detalle	Por qué
Detección automática del símbolo (`symbol_candidates`)	Cada broker llama al oro distinto
Quitar la última vela	Aún se está formando; su "cierre" va a cambiar → sería una mentira
Hora del servidor del broker	Las horas de MT5 están en la zona horaria del broker (normalmente GMT+2/+3)
Importar MT5 dentro de la función	Así el resto del proyecto funciona en Linux (CI/Docker), donde MT5 no existe
Archivo de muestra (`--sample`)	Una copia pequeña (4.000 velas) guardada en Git para las pruebas del CI

🧪 Pruébalo tú

1. En `params.yaml`, cambia `n_bars: 30000` por `50000` y ejecuta `dvc repro`. ¿Qué re-ejecuta DVC? (Pista: *todo*, porque cambiaron los datos crudos.)
2. Cambia `timeframe: H1` por `H4`. ¿Cuántas velas obtienes? ¿Cómo cambia el acierto?

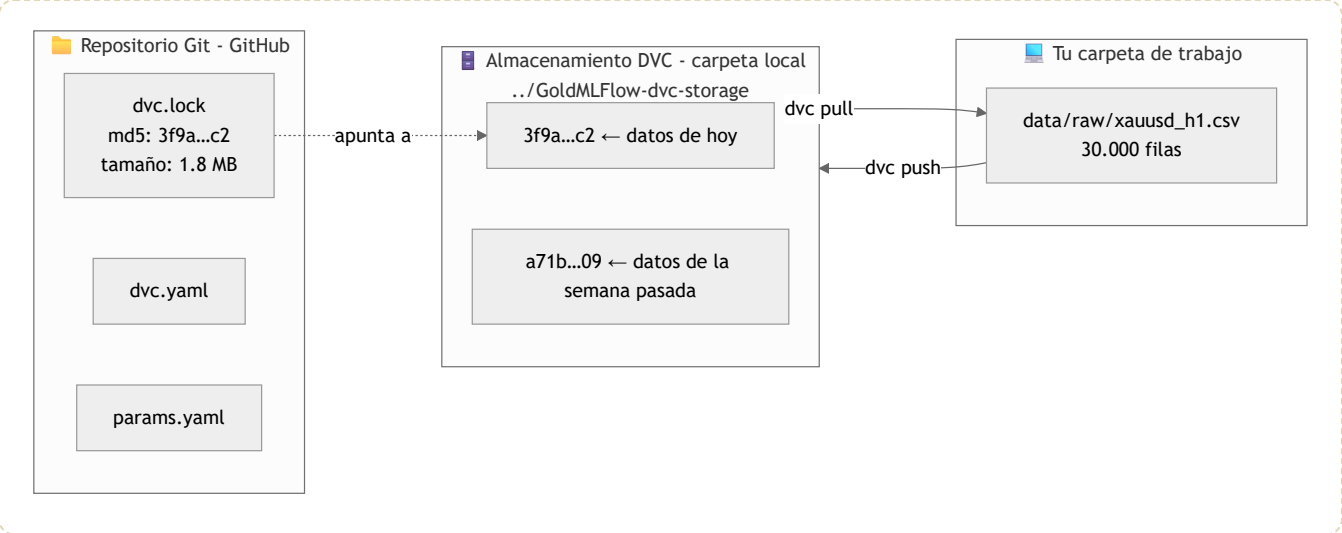
---

03 · Control de versiones de datos (DVC)

🍳 **Analogía**	📸 Fotos con fecha de tu despensa: siempre puedes volver a la de ayer
🎯 **Objetivo**	Versionar archivos de datos grandes + hacer el pipeline reproducible
🏭 **Archivos**	`dvc.yaml` , `dvc.lock` , `.dvc/config` , `params.yaml`
▶️ **Ejecutar**	`dvc repro` , `dvc push` , `dvc pull` , `dvc dag` , `dvc metrics show`

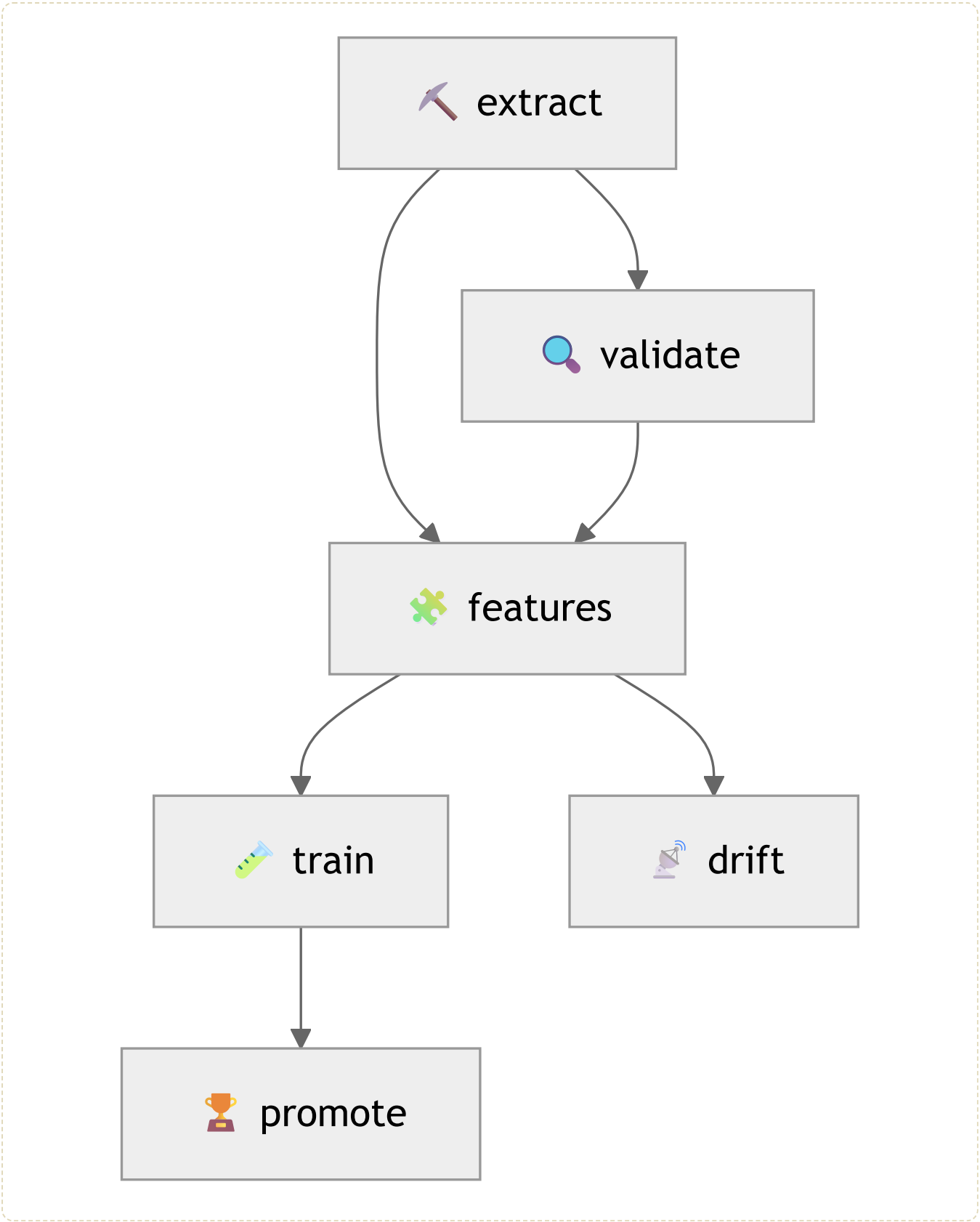
🤔 ¿Por qué no subir el CSV a Git y ya?

Git es excelente para **código** (archivos de texto pequeños) y malo para **datos** (archivos grandes que cambian completos). DVC = **"Git para datos"**: Git guarda una *huella* diminuta (hash) del archivo, y el archivo vive en un **almacenamiento** (aquí, una carpeta local).



🧬 Nuestro pipeline (`dvc.yaml`)

Cada **etapa** declara: el comando, de qué **depende** (código, datos, parámetros) y qué **produce**.



(Es lo que imprime `dvc dag`.)

```
  train:
    cmd: python -m src.models.train
    deps:                          # si alguno cambia → se re-ejecuta
      - src/models/train.py
      - data/processed/features.csv
    params:
      - train                      # solo la sección "train:" de params.yaml
    outs:
      - reports/train_results.json
```

🧠 La magia: `dvc repro` solo re-ejecuta lo que cambió

Si cambias...	DVC re-ejecuta...
`train.models.xgboost.max_depth` en params.yaml	train → promote
`features.rsi_window`	features → train → promote → drift
`src/monitoring/drift.py`	solo drift
nada	nada (*"didn't change, skipping"*)

📸 `dvc.lock` = la foto exacta

Después de cada ejecución, `dvc.lock` guarda el **md5 de cada entrada y salida**. Haz commit en Git → cualquiera puede reproducir **exactamente** el mismo resultado meses después:

```
git checkout <commit-antiguo>   # código + dvc.lock de ese día
dvc pull                        # los datos de ese día
dvc repro                       # los mismos resultados
```

🗄️ Nuestro almacenamiento local

```
dvc remote add -d localstorage ../GoldMLFlow-dvc-storage   # ya configurado en .dvc/config
dvc push      # sube las versiones de datos al almacenamiento
dvc pull      # baja los datos que corresponden al dvc.lock actual
```

> 🏢 En una empresa el almacenamiento sería S3, Google Cloud Storage, Azure Blob o una unidad compartida: solo cambias la URL del remote. Todo lo demás queda igual.

🔀 ¿Qué va a Git y qué a DVC?

Archivo	Dónde	Por qué
`data/raw/xauusd_h1.csv` (1,8 MB, cambia)	**DVC**	datos grandes
`data/processed/features.csv`	**DVC**	datos generados
`data/sample/xauusd_h1_sample.csv` (pequeño)	**Git** (`cache: false`)	el CI lo necesita y no puede llegar al almacenamiento local
`models/champion/` (371 KB)	**Git** (`cache: false`)	el CD construye la imagen Docker a partir de él
`reports/metrics.json`	**Git** (`metrics:`)	comparar ejecuciones con `dvc metrics diff`

🧪 Pruébalo tú

```
dvc dag                       # dibuja el pipeline
dvc metrics show              # métricas actuales
# cambia train.models.xgboost.max_depth a 5 en params.yaml, y luego:
dvc repro
dvc metrics diff              # compara con el último commit
dvc params diff               # qué parámetros cambiaron
```

---

04 · Validar datos y construir features

🍳 **Analogía**	🔍 Revisar que los ingredientes no estén podridos y luego 🔪 picarlos
🎯 **Objetivo**	Garantizar datos sanos y convertir velas en "pistas" numéricas + la respuesta
📓 **Notebooks**	`02_validate_explore`, `03_features`
🏭 **Código de producción**	`src/data/validate.py`, `src/features/build.py`
✅ **Pruebas**	`tests/test_data.py`, `tests/test_features.py`

Parte A · 🔍 Validación: "basura entra, basura sale"

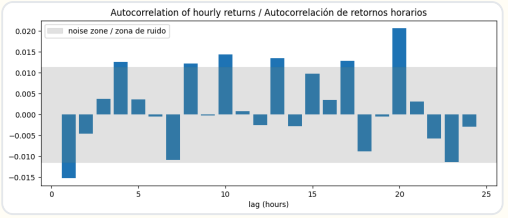
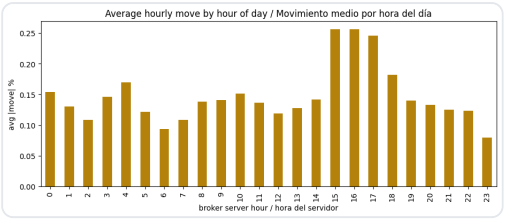
Antes de que cualquier modelo vea los datos, 8 preguntas de sí/no:

Chequeo	Detecta...
required_columns	una exportación rota
enough_rows (≥ 1.000)	MT5 devolvió casi nada
no_missing_values	celdas vacías
no_duplicated_timestamps	la misma hora dos veces
time_is_sorted	filas desordenadas
prices_positive	precios en cero o negativos
high_is_highest / low_is_lowest	velas imposibles

Si **cualquiera** falla → el script termina con código 1 → **DVC detiene el pipeline**. Ningún modelo se entrena con datos rotos.

```
✅ required_columns    ✅ enough_rows    ✅ no_missing_values    ✅ no_duplicated_timestamps
✅ time_is_sorted      ✅ prices_positive    ✅ high_is_highest    ✅ low_is_lowest
Data validation passed (29999 rows)
```

🔎 Lo que nos enseñó explorar los datos

Hallazgo	Gráfico	Consecuencia
SUBE 50,9 % / BAJA 49,1 %	—	"Siempre SUBE" ya acierta 50,9 % → ese es el **baseline a vencer**
Los retornos horarios casi no se relacionan con el pasado		Espera ~51–53 %, no 90 %
El oro se mueve más en la apertura de Londres y Nueva York		Darle al modelo la **hora del día**

Parte B · 🧩 Features: convertir velas en pistas

Un modelo no sabe leer un gráfico. Para cada vela *t* calculamos números que describen **solo el pasado**, más el **objetivo (target)**:

```
vela t-200 … t-1, t   ──▶   pistas(t)   ──▶ 🤖 ──▶  ¿la vela t+1 cerrará más arriba?
   (solo el PASADO)                                  target = 1 (SUBE) / 0 (BAJA)
```

Familia	Pistas	En palabras simples
🏃 Momentum	`ret_1h, ret_2h, ret_3h, ret_5h, ret_10h, ret_24h`	¿Cuánto se movió el precio en las últimas N horas?
📈 Tendencia	`dist_sma_10, dist_sma_50, dist_sma_200`	¿Está el precio sobre o bajo su promedio?
🎚️ Osciladores	`rsi`, `macd_hist`	¿Está "cansado" de subir o bajar?
🌪️ Riesgo	`volatility`, `range_pct`, `body_pct`	¿Qué tan nervioso está el mercado?
📊 Actividad	`volume_z`	¿Hay actividad inusual?
🕐 Calendario	`hour_sin, hour_cos, dayofweek`	¿Asia? ¿Londres? ¿Nueva York? ¿Viernes?

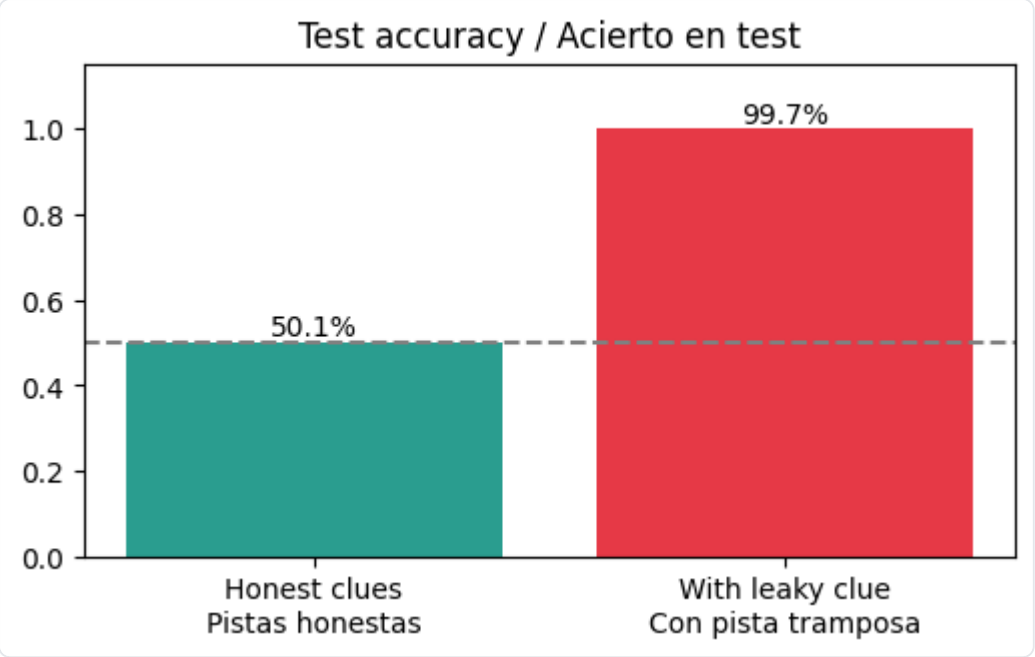
Resultado: 29.799 filas × 18 pistas + objetivo.

> 🕐 **¿Por qué seno/coseno para la hora?** Para un modelo, 23:00 y 00:00 parecen lejanas (23 vs 0), pero son vecinas. Poner las horas en un círculo lo resuelve.

🚨 Fuga de datos (data leakage): la trampa #1

Fuga = una pista que sin querer contiene información del futuro. En el papel el modelo parece un genio; en la vida real pierde dinero, porque en tiempo real el futuro todavía no existe.

En el notebook 03 agregamos una pista de apariencia inocente: una media móvil **centrada** en la vela (promedia la hora anterior, la actual… **y la siguiente**):



50,1 % → 99,7 % con una sola línea de código. **Si los resultados parecen demasiado buenos para ser verdad, es que no lo son.**

🛡️ Cómo nos protege producción

```
def test_no_leakage_features_ignore_the_future(sample_raw, params):
    """Cambiar precios FUTUROS no debe cambiar las pistas de hoy."""
    base = make_features(sample_raw, ...)
    tampered = sample_raw.copy()
    tampered.loc[últimas 100 filas, precios] *= 1.5     # 🔨 cambiamos el futuro
    after = make_features(tampered, ...)
    assert pistas de las filas anteriores idénticas      # ✅ o el CI falla
```

Esta prueba corre en **cada push** en GitHub Actions.

♻️ El mismo código para entrenar y para servir

```
make_features(df, params, with_target=True)   # entrenamiento: agrega target, quita la última vela (sin futuro aún)
make_features(df, params, with_target=False)  # API: mismas pistas, conserva la última vela para predecirla
```

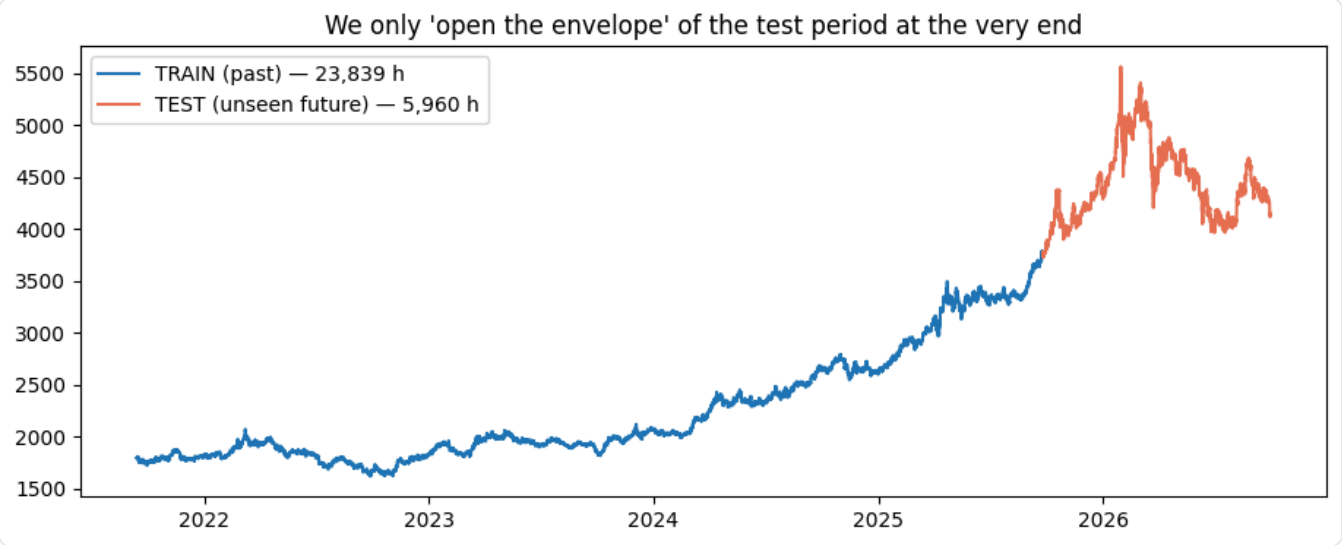
Usar **una sola función** en ambos lados evita el *training/serving skew* (que la API calcule las pistas un poco distinto que el entrenamiento → errores silenciosos).

05 · Entrenar modelos y registrarlos con MLflow

🍳 **Analogía**	👩‍🍳 Probar varias recetas y 📒 anotarlo todo en un cuaderno de laboratorio
🎯 **Objetivo**	Comparar modelos de forma justa y no perder nunca un resultado
📓 **Notebook**	`04_train_mlflow`
🏭 **Código de producción**	`src/models/train.py`, `src/models/tracking.py`
▶️ **Ejecutar**	`dvc repro train` · Interfaz: `mlflow ui --backend-store-uri sqlite:///mlflow.db`

⏳ Regla #1 en series de tiempo: nunca mezclar

```
|■■■■■■■■■■■■■■■■■■■■■■ TRAIN 80 % (pasado) ■■■■■■■■■■■■■■■■■■■■■■|□□□ TEST 20 % (futuro no visto) □□□|
 2021-09-13 ──────────────────────────────────────────────── 2025-09-24 ──────────────────── 2026-09-29
            23.839 horas                                              5.960 horas
```



Dentro de la parte de entrenamiento usamos **validación cruzada walk-forward** (`TimeSeriesSplit`): "exámenes de práctica" que siempre entrenan con el pasado y prueban con lo que viene después.

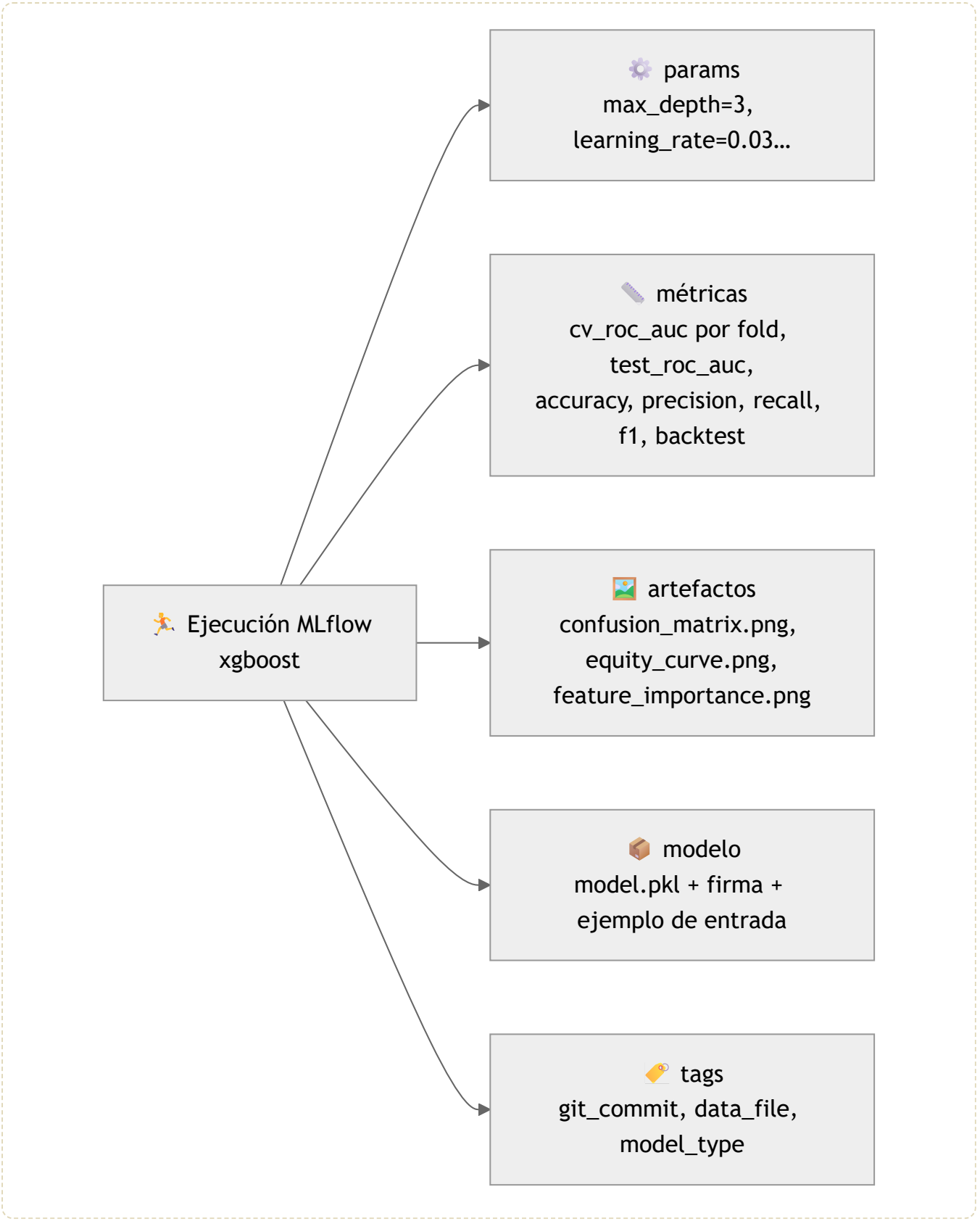
```
fold 1: ▓▓▓▒
fold 2: ▓▓▓▓▒
fold 3: ▓▓▓▓▓▒          ▓ = entrenar   ▒ = validar
fold 4: ▓▓▓▓▓▓▒
fold 5: ▓▓▓▓▓▓▓▒
```

🤖 Los 4 concursantes

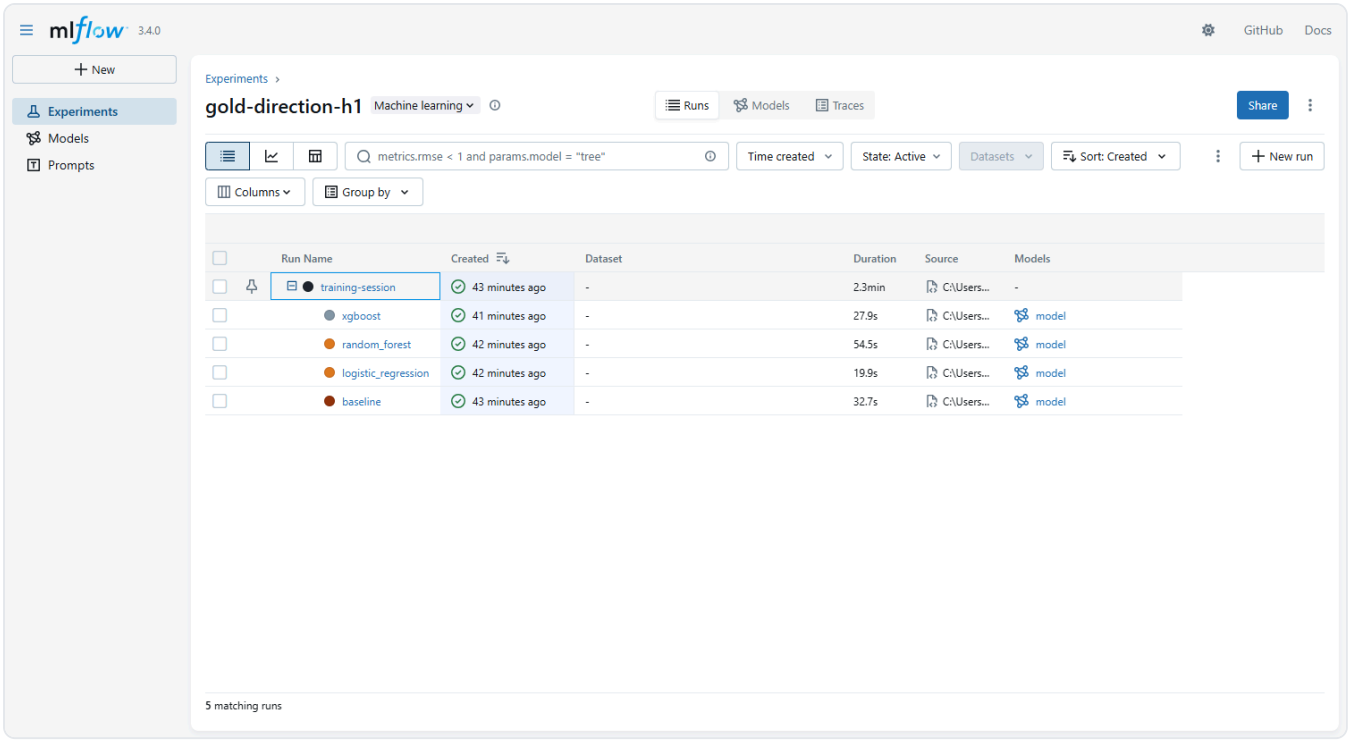
Modelo	Idea	Por qué está aquí
baseline (`DummyClassifier`)	siempre dice la respuesta más común (SUBE)	🪙 la "moneda al aire" que todos deben vencer
logistic_regression	una suma ponderada de las pistas	simple, rápida, difícil de sobreajustar
random_forest	300 árboles de decisión votan	capta patrones no lineales
xgboost	árboles que corrigen los errores de los anteriores	suele ser el más fuerte en datos tabulares

Cada modelo es un `Pipeline` de sklearn (p. ej. escalador + modelo), así el preprocesamiento **viaja junto al modelo** hasta la API.

📒 Lo que MLflow registra de cada modelo

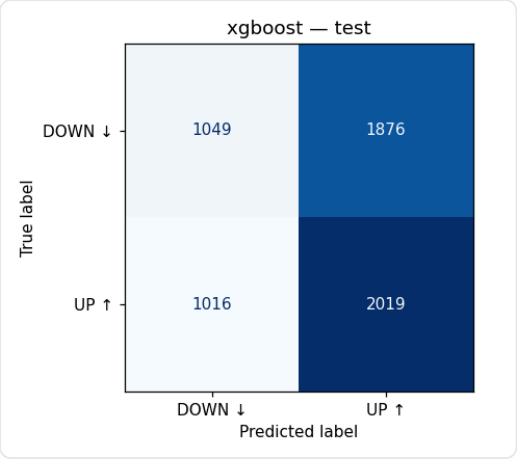
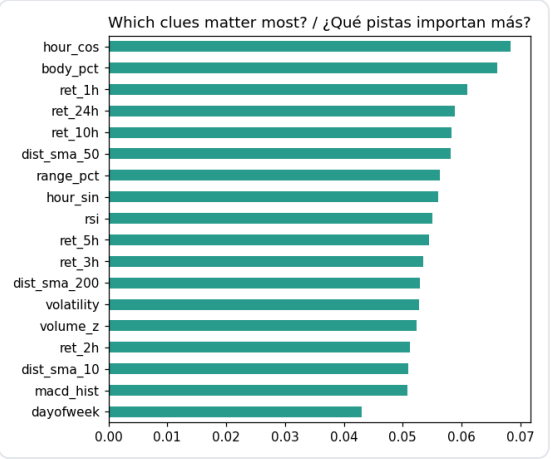


Las ejecuciones están **anidadas**: una ejecución padre `training-session` con una hija por modelo.



🖼️ Gráficos guardados dentro de cada ejecución



Matriz de confusión	Importancia de las pistas
	

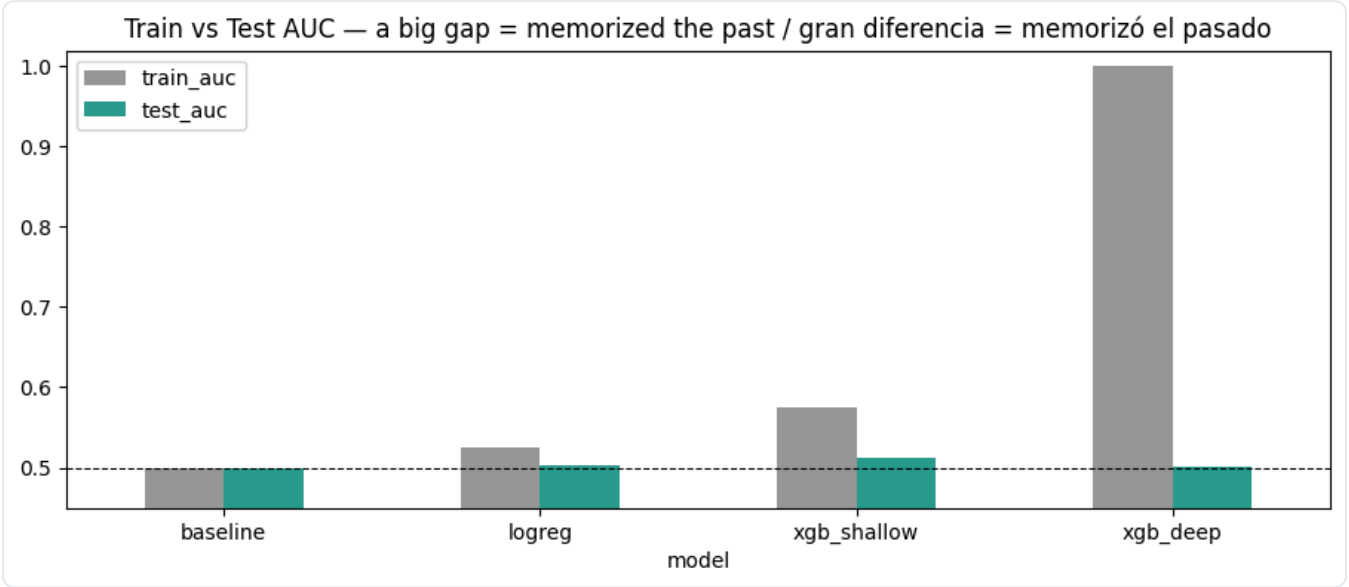
✍️ La "firma" (signature): el contrato del modelo

MLflow guarda qué columnas (y de qué tipo) entran y qué sale. Si alguien después envía columnas equivocadas, MLflow avisa en lugar de predecir basura en silencio.

📊 Resultados (test = los últimos 12 meses, nunca vistos)

Modelo	ROC-AUC test	accuracy test
🥇 **xgboost**	**0,5140**	**51,48 %**
random_forest	0,5032	50,45 %
logistic_regression	0,5028	50,45 %
baseline 🪙	0,5000	50,92 %

🎓 Lección del notebook: sobreajuste (overfitting)



`xgb_deep` (8 niveles de profundidad) saca **0,999 de AUC en el pasado** pero **0,501 en el futuro**: **memorizó ruido**. En mercados ruidosos los modelos simples generalizan mejor.

🗂️ ¿Dónde se guardan los datos de MLflow?

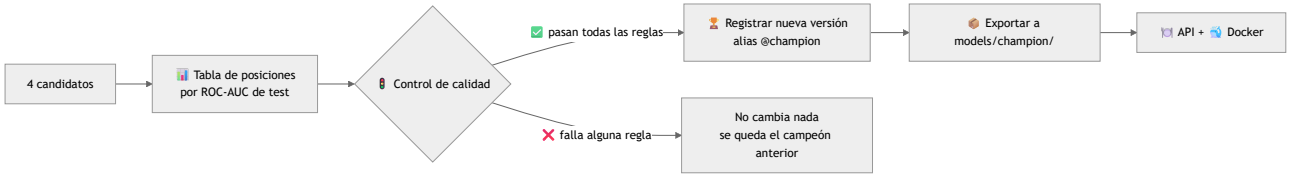
Modo	Cómo	Uso
Local (por defecto)	`mlflow.db` (SQLite) + carpeta `mlartifacts/`	aprender, una persona
Equipo	`docker compose --profile team up mlflow` + `$env:MLFLOW_TRACKING_URI="http://localhost:5000"`	un servidor compartido para todos

---

06 · Evaluar, control de calidad y Registro de Modelos

🍳 **Analogía**	🏅 El jurado prueba cada plato; solo el mejor entra al menú
🎯 **Objetivo**	Promover un modelo **solo** si es objetivamente mejor que adivinar
📓 **Notebook**	`05_evaluate_registry`
🏭 **Código de producción**	`src/models/promote.py` , `gate.py` , `check_gate.py`

🚦 El flujo



📏 Métricas en palabras simples

Métrica	Pregunta que responde	Nuestro campeón
Accuracy	% de horas en que acertó SUBE/BAJA	51,48 %
Precision	Cuando dice SUBE, ¿cuántas veces sube de verdad?	51,84 %
Recall	De todas las subidas reales, ¿cuántas detectó?	66,52 %
ROC-AUC	¿Ordena "más probable que suba" por encima de "menos probable"? 0,5 = moneda	0,5140

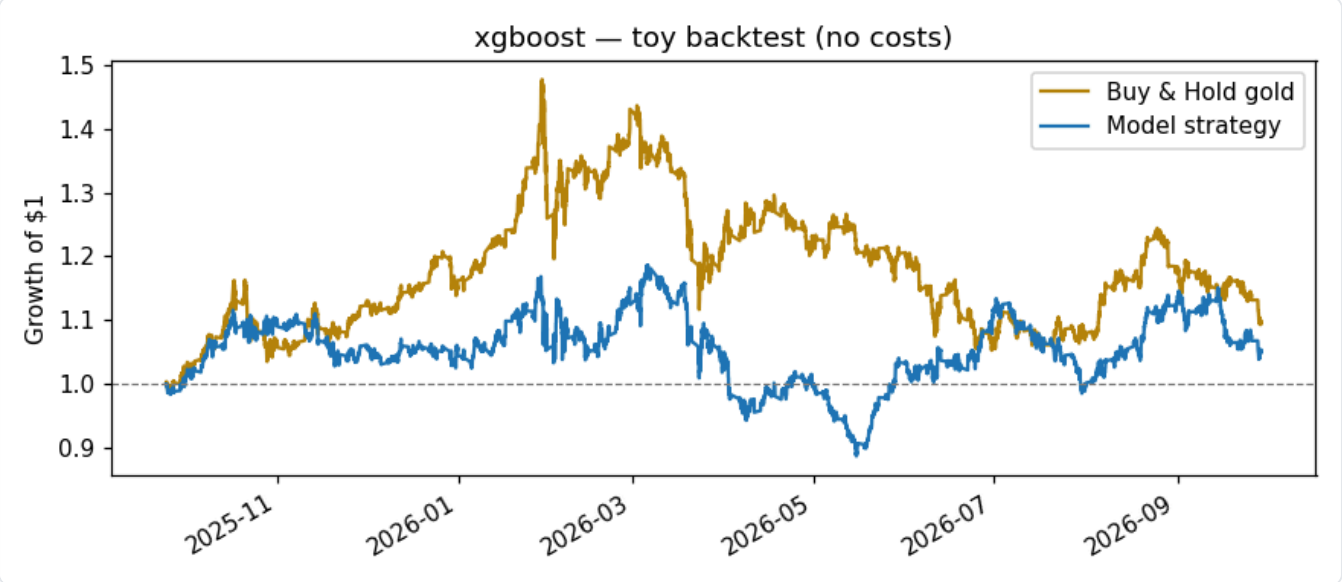
🚦 El control de calidad: reglas escritas, no opiniones

Definido en `params.yaml → evaluate`:

```
evaluate:
  primary_metric: roc_auc
  min_roc_auc: 0.51          # debe ser mejor que una moneda por un margen mínimo
  must_beat_baseline: true   # debe vencer a "siempre SUBE" en AUC Y en accuracy
```

```
Leaderboard (test_roc_auc):
  1. xgboost              test_roc_auc=0.5140  acc=0.5148
  2. random_forest        test_roc_auc=0.5032  acc=0.5045
  3. logistic_regression  test_roc_auc=0.5028  acc=0.5045
  4. baseline             test_roc_auc=0.5000  acc=0.5092
  ✅ roc_auc >= 0.51
  ✅ beats baseline (roc_auc)
  ✅ beats baseline (accuracy)
🏆 Registered gold-direction-h1 v2 → alias @champion
```

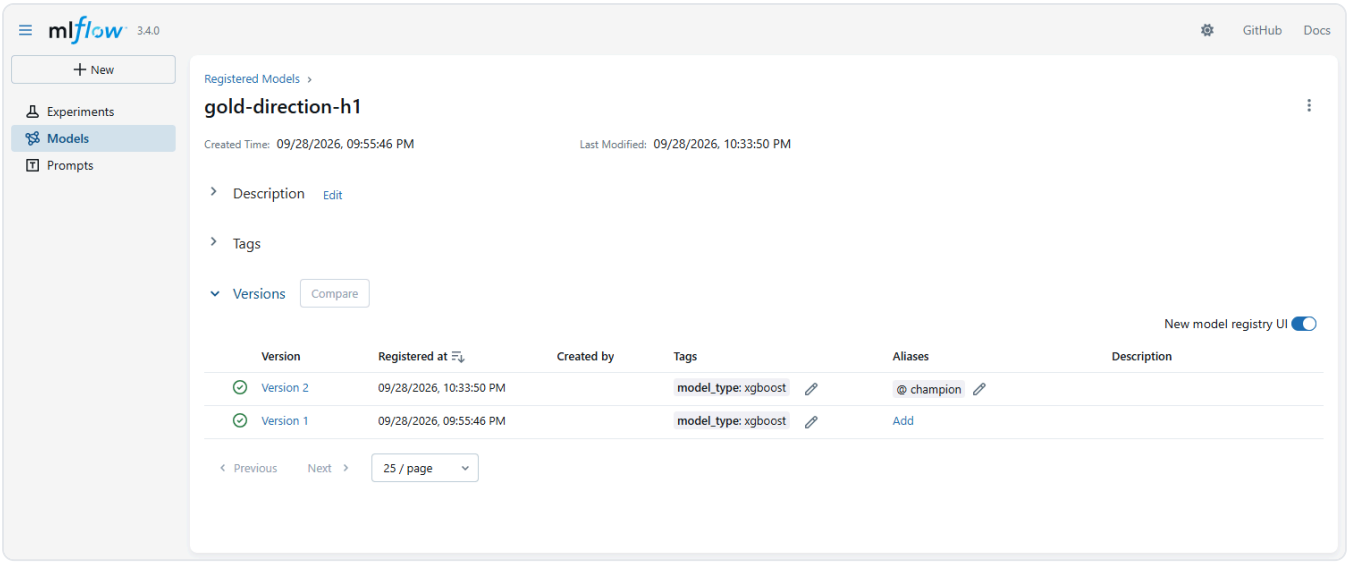
💰 La verdad honesta: superar el 50 % ≠ ganar dinero



Estrategia de juguete (comprar si dice SUBE, vender si dice BAJA, **sin comisiones**): **+5,3 %**. Simplemente mantener oro: **+9,8 %**. El modelo es *un poco* mejor que una moneda para predecir la dirección, pero eso no lo convierte en un buen sistema de trading. Este proyecto trata del **proceso**.

🏆 El Registro de Modelos (Model Registry)

Como el catálogo de una biblioteca: cada modelo aprobado recibe un **número de versión**, y un **alias** (`@champion`) apunta al que está en producción.




```
# Cargar el modelo de producción POR NOMBRE, sin rutas de archivos:
model = mlflow.sklearn.load_model("models:/gold-direction-h1@champion")
```

Acción	Cómo
Promover	mover `@champion` a la nueva versión (lo hace `promote.py` automáticamente)
Volver atrás (rollback)	mover `@champion` a una versión anterior (1 clic en la interfaz)
Auditar	cada versión enlaza a su ejecución: params, métricas, gráficos, commit de git

📦 ¿Por qué además exportar a `models/champion/`?

El robot de GitHub (CD) construye la imagen Docker pero **no puede llegar a tu MLflow local**. Por eso el campeón también se copia al repositorio (371 KB): `model/` (formato MLflow) + `metadata.json` (métricas, pistas, versión).

🛡️ El control de calidad también corre en el CI

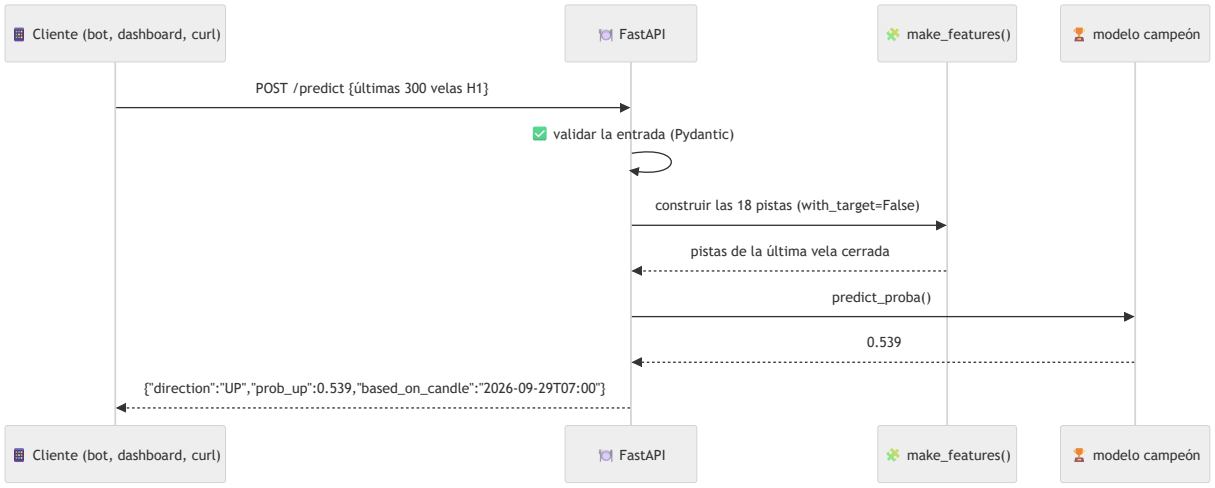
`python -m src.models.check_gate` corre en **cada push**: si alguien sube un campeón que no cumple las reglas, el build queda ❌ rojo y no se puede fusionar.

---

07 · API del modelo (FastAPI)

🍳 **Analogía**	🧑‍🍳 El mesero: toma los pedidos (peticiones) y trae el plato (predicción)
🎯 **Objetivo**	Que **cualquier** programa pregunte "¿SUBE o BAJA?" por HTTP
📓 **Notebook**	`06_serve_api`
🏭 **Código de producción**	`src/api/main.py`, `src/api/schemas.py`
▶️ **Ejecutar**	`uvicorn src.api.main:app` → http://localhost:8000/docs

🔄 El viaje de una predicción

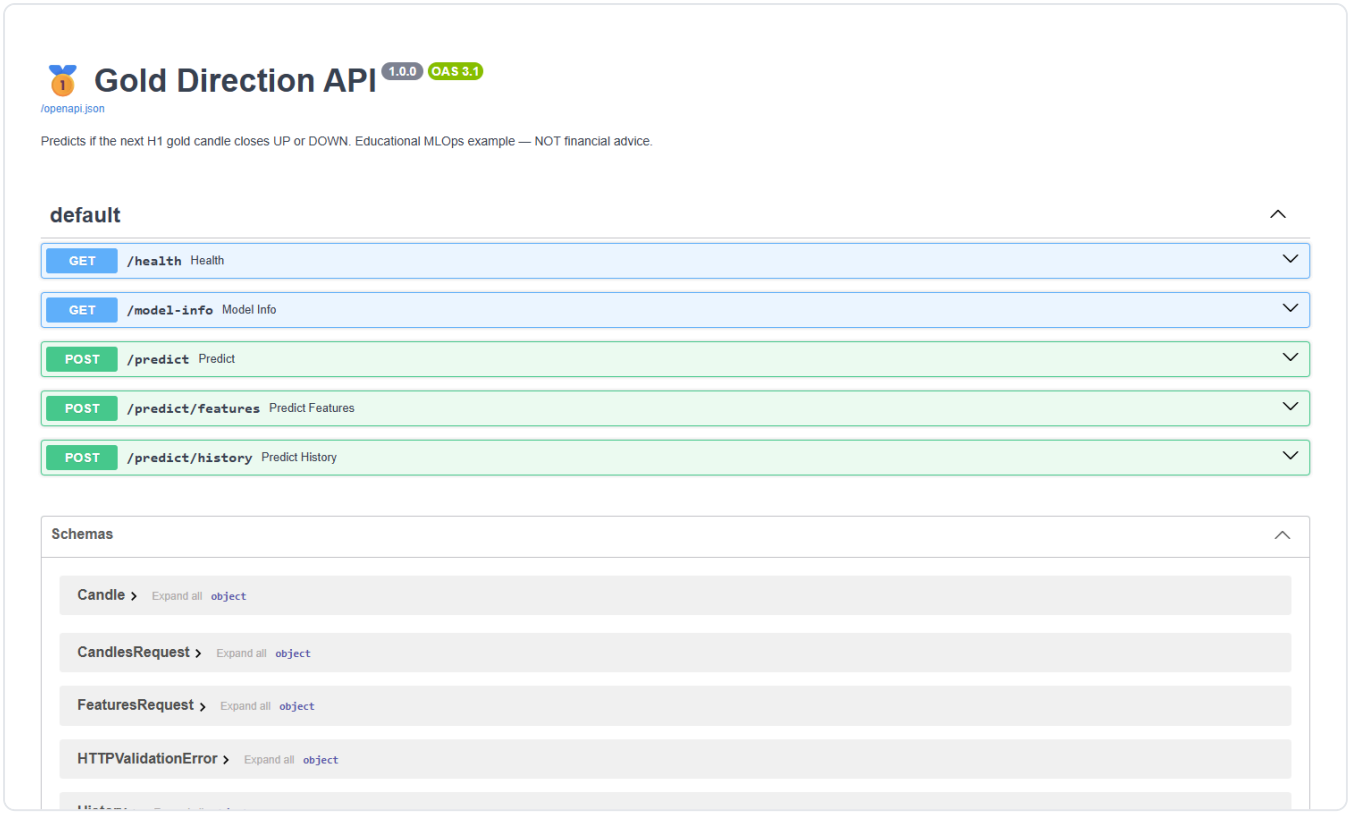


🚪 Endpoints

Método	Ruta	Para qué
GET	`/health`	"¿Estás vivo?" (lo usa el healthcheck de Docker)
GET	`/model-info`	Qué modelo, versión, métricas y pistas
POST	`/predict`	Envía ≥ 250 velas → SUBE/BAJA para la **próxima** hora
POST	`/predict/features`	Envía las 18 pistas directamente (uso avanzado)
POST	`/predict/history`	P(SUBE) para **cada** vela enviada (lo usan los gráficos del dashboard)

📖 Documentación interactiva gratis (Swagger)

FastAPI genera esta página automáticamente a partir del código. Haz clic en **"Try it out"** para llamar a la API desde el navegador:



💻 Cómo llamarla

```
# PowerShell
Invoke-RestMethod http://localhost:8000/health
Invoke-RestMethod http://localhost:8000/model-info
```

```
# Python
import httpx, pandas as pd
candles = pd.read_csv("data/sample/xauusd_h1_sample.csv").tail(300)
r = httpx.post("http://localhost:8000/predict", json={"candles":
candles.to_dict(orient="records")})
print(r.json())   # {'direction': 'DOWN', 'prob_up': 0.452, ...}
```

```
# 🔮 El ciclo completo en vivo: MT5 → API → predicción
python scripts/live_demo.py
```

```
╔══════════════════════════════════════════════╗
  🥇 GOLD H1 — next candle prediction
  Last closed candle : 2026-09-29T07:00:00   close=4126.53
  Model              : xgboost v2  (test AUC 0.514)
  Prediction         : 🟢 ▲ UP / SUBE
  Probability UP     : 53.9%
╚══════════════════════════════════════════════╝
```

Buenas prácticas incluidas

Práctica	Dónde
Validación de entrada (Pydantic): ≥ 250 velas, precios > 0 → error 422 claro, nunca una caída	`schemas.py`
Modelo cargado **una vez** al iniciar, no en cada petición	`lifespan()`
Ubicación del modelo configurable (variable `MODEL_DIR`)	`main.py`
El mismo código de features que el entrenamiento (sin training/serving skew)	`make_features(with_target=False)`
Pruebas con un cliente en memoria (`TestClient`) en cada push	`tests/test_api.py`

---

08 · Contenedores (Docker)

🍳 **Analogía**	🚚 Un food-truck: toda la cocina (Python, librerías, código, modelo) empacada para estacionarse en cualquier lugar
🎯 **Objetivo**	Acabar con el "en mi máquina funciona": la misma caja corre en tu PC, en el CI y en la nube
🏭 **Archivos**	`Dockerfile`, `Dockerfile.dashboard`, `docker-compose.yml`, `.dockerignore`
▶️ **Ejecutar**	`docker compose up --build`

📦 Imagen vs contenedor

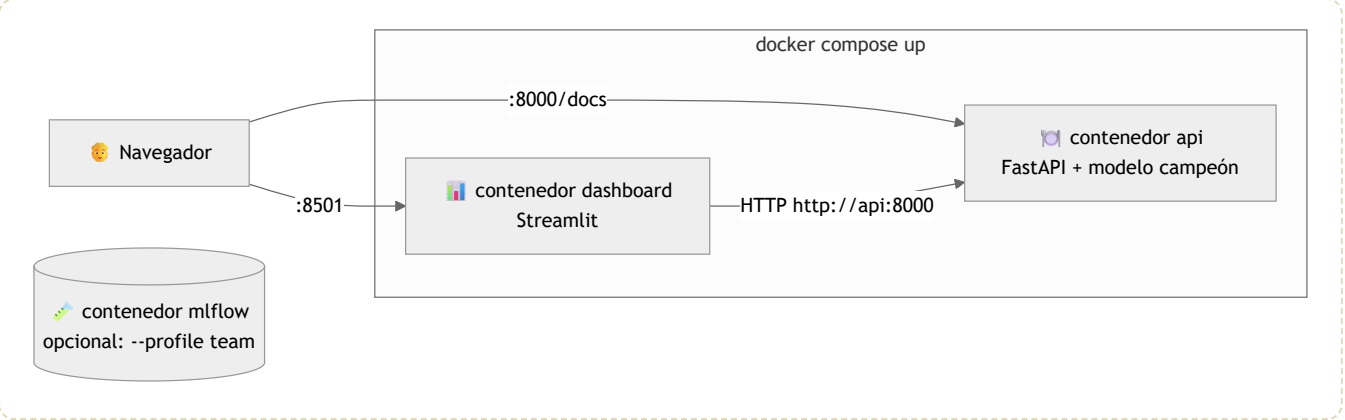
Concepto	Analogía	Comando
Dockerfile	la receta para construir el camión	—
Imagen	el camión terminado, estacionado (solo lectura)	`docker build`
Contenedor	el camión **abierto y atendiendo**	`docker run`
Registry (ghcr.io)	el estacionamiento donde se comparten los camiones	`docker push / pull`
Compose	una flota de camiones que trabajan juntos	`docker compose up`

🧱 Nuestra imagen de la API, capa por capa

```
FROM python:3.12-slim                          # 1. un Linux pequeño + Python
COPY requirements-api.txt .
RUN pip install -r requirements-api.txt        # 2. librerías (capa cacheada → builds rápidos)
COPY src/ ... models/champion/ ...             # 3. solo el código que la API necesita + el modelo
RUN useradd appuser && USER appuser            # 4. no correr como root (seguridad)
HEALTHCHECK ... /health                        # 5. Docker revisa que esté vivo cada 30 s
CMD ["uvicorn", "src.api.main:app", ...]       # 6. lo que corre al iniciar el contenedor
```

> 💡 **¿Por qué XGBoost es `xgboost-cpu` en Linux?** El paquete normal descarga ~350 MB de librerías de GPU de NVIDIA que no usamos. `xgboost-cpu` mantiene las imágenes pequeñas.

🚢 Dos contenedores trabajando juntos



- Los contenedores se encuentran **por el nombre del servicio** (`http://api:8000`).
- El dashboard **espera** a que la API esté sana (`depends_on: condition: service_healthy`).
- El dashboard nunca carga el modelo por su cuenta: se lo pregunta a la API, como cualquier cliente real.

```
docker compose up --build        # construir + iniciar ambos
docker compose ps                # estado (la api debe decir "healthy")
docker compose logs -f api       # logs en vivo
docker compose down              # detener y eliminar
```

> ⚠️ **¿Puerto ocupado?** `$env:API_PORT=8010; $env:DASHBOARD_PORT=8502; docker compose up`

✅ Prueba de que es reproducible

Las mismas 300 velas enviadas a la API **en Windows** y **dentro del contenedor Linux** dan exactamente la misma respuesta:

```
local (Windows) : {'direction': 'DOWN', 'prob_up': 0.452, 'based_on_candle': '2026-09-
29T05:00:00'}
docker (Linux)  : {'direction': 'DOWN', 'prob_up': 0.452, 'based_on_candle': '2026-09-
29T05:00:00'}
```

🟡 ¿Por qué MetaTrader 5 no está en un contenedor?

El paquete `MetaTrader5` solo funciona en **Windows con la terminal abierta**, y las imágenes Docker son Linux. Por eso la **extracción corre en tu PC** y todo lo que viene después usa los datos versionados con DVC. Esta separación es común en la vida real (fuentes de datos que solo viven en un lugar).

---

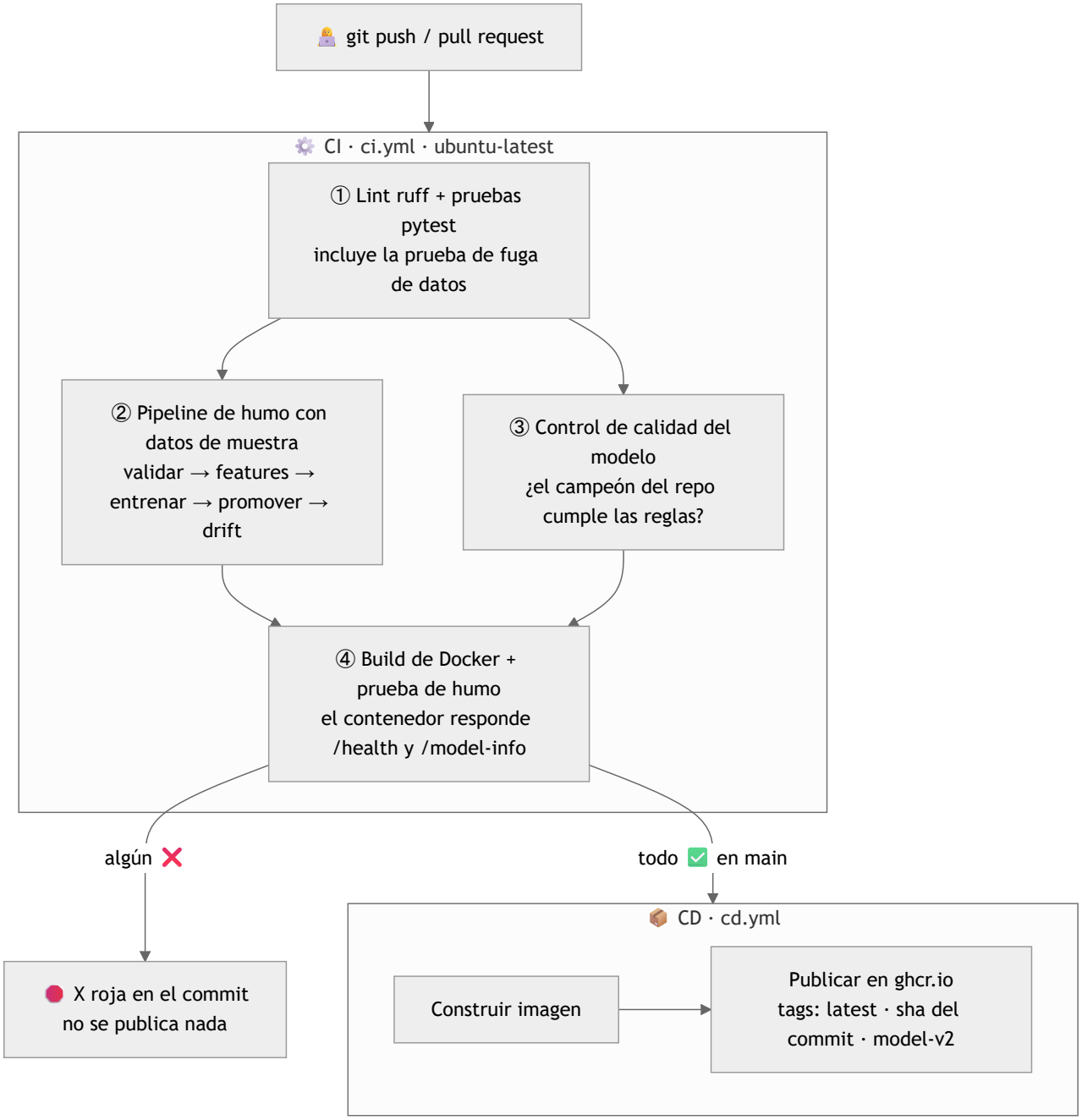
09 · CI/CD con GitHub Actions

🍳 **Analogía**	👮 El inspector de sanidad revisa cada cambio **antes** de que el restaurante abra con él; luego 🚚 el camión nuevo se entrega automáticamente
🎯 **Objetivo**	Cada push se prueba solo; solo los cambios en verde se publican
🏭 **Archivos**	`.github/workflows/ci.yml`, `.github/workflows/cd.yml`
🌐 **Verlo**	https://github.com/MirandaCR/GoldMLFlow/actions

🤔 CI vs CD

	CI: Integración Continua	**CD: Entrega Continua**
Pregunta	"¿Este cambio es seguro?"	"¡Publícalo!"
Cuándo	en cada push y pull request	solo cuando el CI está ✅ en `main`
Hace	lint, pruebas, pipeline, control de calidad, build de Docker	construye y publica la imagen Docker
Resultado	✅ / ❌ junto a cada commit	`ghcr.io/mirandacr/goldmlflow:latest`

El flujo completo del robot



🔎 Cada trabajo del CI en palabras simples

Trabajo	Qué detecta
① **Lint y pruebas**	código desordenado, funciones rotas, **fuga de datos**, errores de la API
② **Pipeline de humo**	"todo el pipeline sigue corriendo de punta a punta" (con la muestra de 4.000 velas en Git: el robot no tiene MT5 ni acceso a tu almacenamiento DVC local)
③ **Control de calidad**	alguien subió un **modelo peor** de lo que permiten las reglas
④ **Build de Docker**	la imagen no se construye o el contenedor no responde

Los resultados aparecen en el **Summary** del trabajo (tabla de métricas + chequeos del campeón) y se pueden descargar los artefactos (reportes, ejecuciones de MLflow).

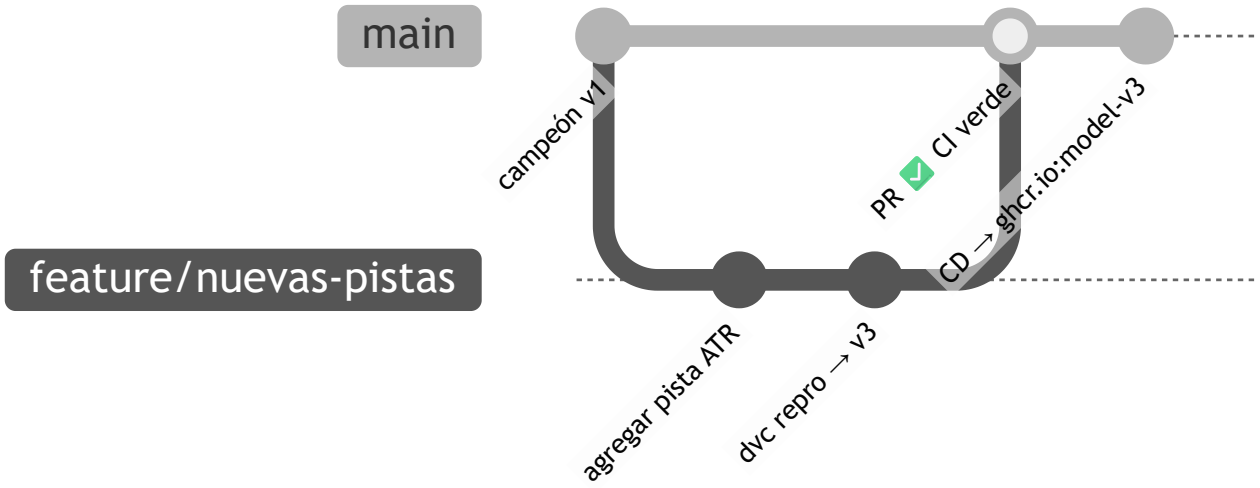
📦 CD: publicar la imagen

Cuando el CI está en verde en `main`, `cd.yml`: 1. lee la versión del campeón en `models/champion/metadata.json` 2. construye la imagen y la sube al **GitHub Container Registry** con 3 tags: `latest`, el SHA del commit y `model-v2` 3. cualquiera puede ejecutarla sin clonar nada:

```
docker run -p 8000:8000 ghcr.io/mirandacr/goldmlflow:latest
```

> 🔐 No hacen falta secretos: el `GITHUB_TOKEN` integrado tiene permiso `packages: write` (declarado en el workflow). La primera vez el paquete es **privado**; hazlo público en GitHub → tu perfil → Packages → goldmlflow → Package settings.

🧭 El día a día con CI/CD



1. Crea una rama, cambia código o parámetros, `dvc repro`, `dvc push`.
2. `git push` → abre un Pull Request → corre el CI.
3. ✅ verde → fusionar → el CD publica la nueva imagen automáticamente.

💡 Para ir más allá (mejoras del mundo real)

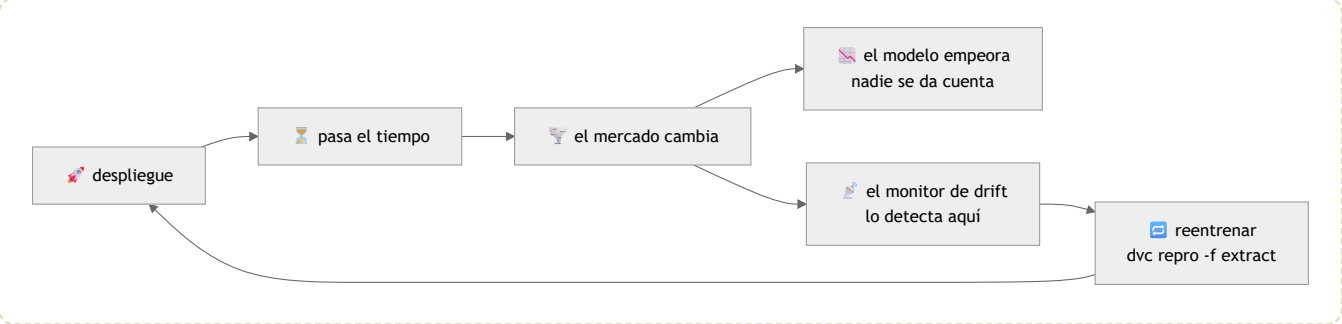
Mejora	Cómo
El CI entrena con **todos** los datos	usar un remote DVC en la nube (S3/GCS) + `dvc pull` en el CI con secretos
Reentrenar de forma programada	agregar `on: schedule: - cron: "0 6 * * 1"`
Comentar métricas en el PR	CML (`cml comment create`)
Desplegar en un servidor	después del push, un paso `ssh` / Cloud Run / Kubernetes

---

10 · Monitoreo y data drift

🔍 **Analogía**	📋 Revisar si los clientes siguen pidiendo los mismos platos que cuando diseñaste el menú
🎯 **Objetivo**	Detectar cuándo el mercado de hoy ya no se parece al que usó el modelo para aprender
📓 **Notebook**	`07_monitor_drift`
🏭 **Código de producción**	`src/monitoring/drift.py` (última etapa de DVC)

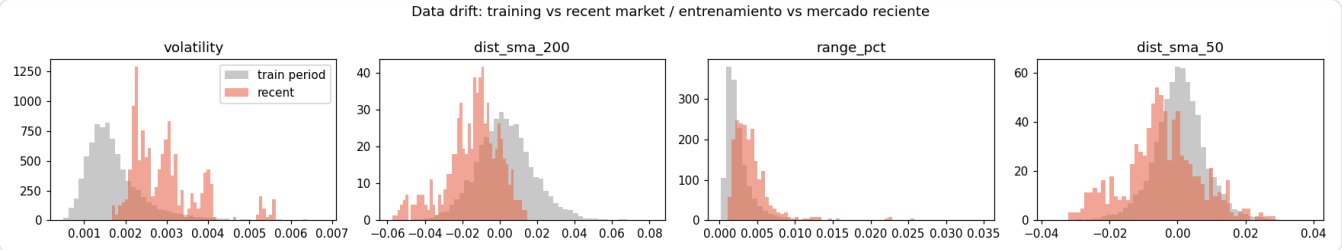
📉 Por qué los modelos empeoran en silencio



Un modelo aprende patrones de un **periodo**. Si el mercado cambia (nuevo régimen de volatilidad, crisis, rally), esos patrones pueden dejar de funcionar. Eso es **data drift**.

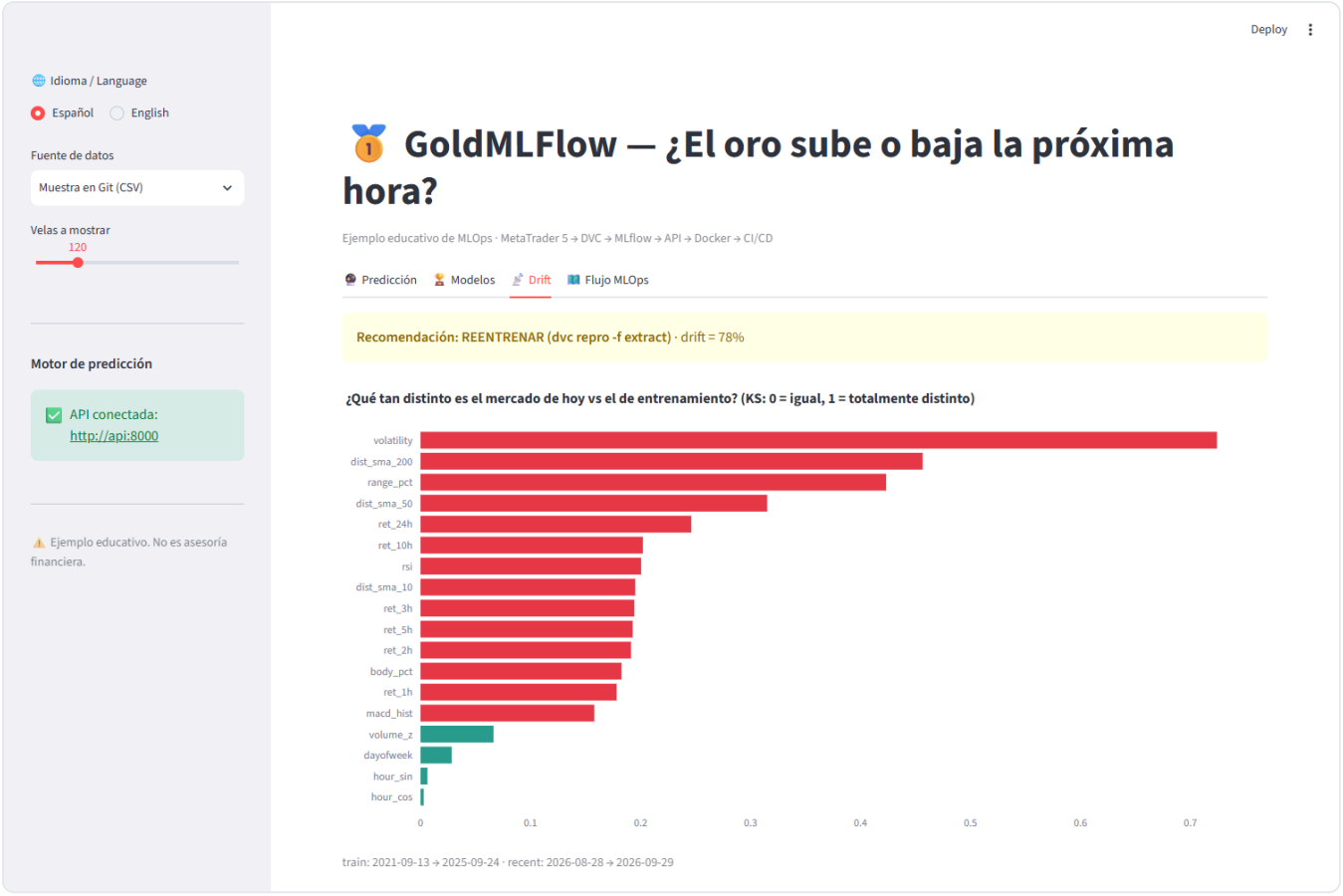
📏 Cómo lo medimos: la prueba KS

Para cada pista comparamos dos histogramas, **periodo de entrenamiento** vs **últimas 500 horas**, con la prueba de **Kolmogorov–Smirnov**: un número de **0 (idénticos) a 1 (totalmente distintos)**.



🔎 Lo que encontramos (datos reales)

Pista	KS	Significado
`volatility`	**0,72**	el oro está **mucho** más volátil ahora que en la mayor parte de 2021–2025
`dist_sma_200`	0,46	el precio está lejos de su promedio de largo plazo (gran rally/corrección)
`range_pct`	0,42	las velas son más grandes
`hour_sin`, `hour_cos`, `dayofweek`	~0,00–0,03	las pistas de calendario no derivan (¡obvio!)



🔁 Qué hacer cuando salta la alarma

```
dvc repro -f extract     # descarga velas nuevas → se re-ejecuta TODO el pipeline
dvc push                 # guarda la nueva versión de datos
git add dvc.lock models/champion reports && git commit -m "retrain: datos nuevos"
git push                 # el CI lo revisa y el CD lo publica
```

Un nuevo campeón solo se promueve si **pasa el control de calidad**; si no, se queda el anterior.

💡 Siguiente nivel

- Ejecutar la revisión de drift **de forma programada** (GitHub Actions `schedule:` o el Programador de tareas de Windows) y enviar una alerta (email/Slack/Telegram).
- Monitorear también el **desempeño**: guardar cada predicción en vivo y compararla con lo que realmente pasó (el "acierto en la ventana" del dashboard lo muestra de forma visual).
- Herramientas: Evidently, WhyLabs, Arize, Prometheus + Grafana.

---

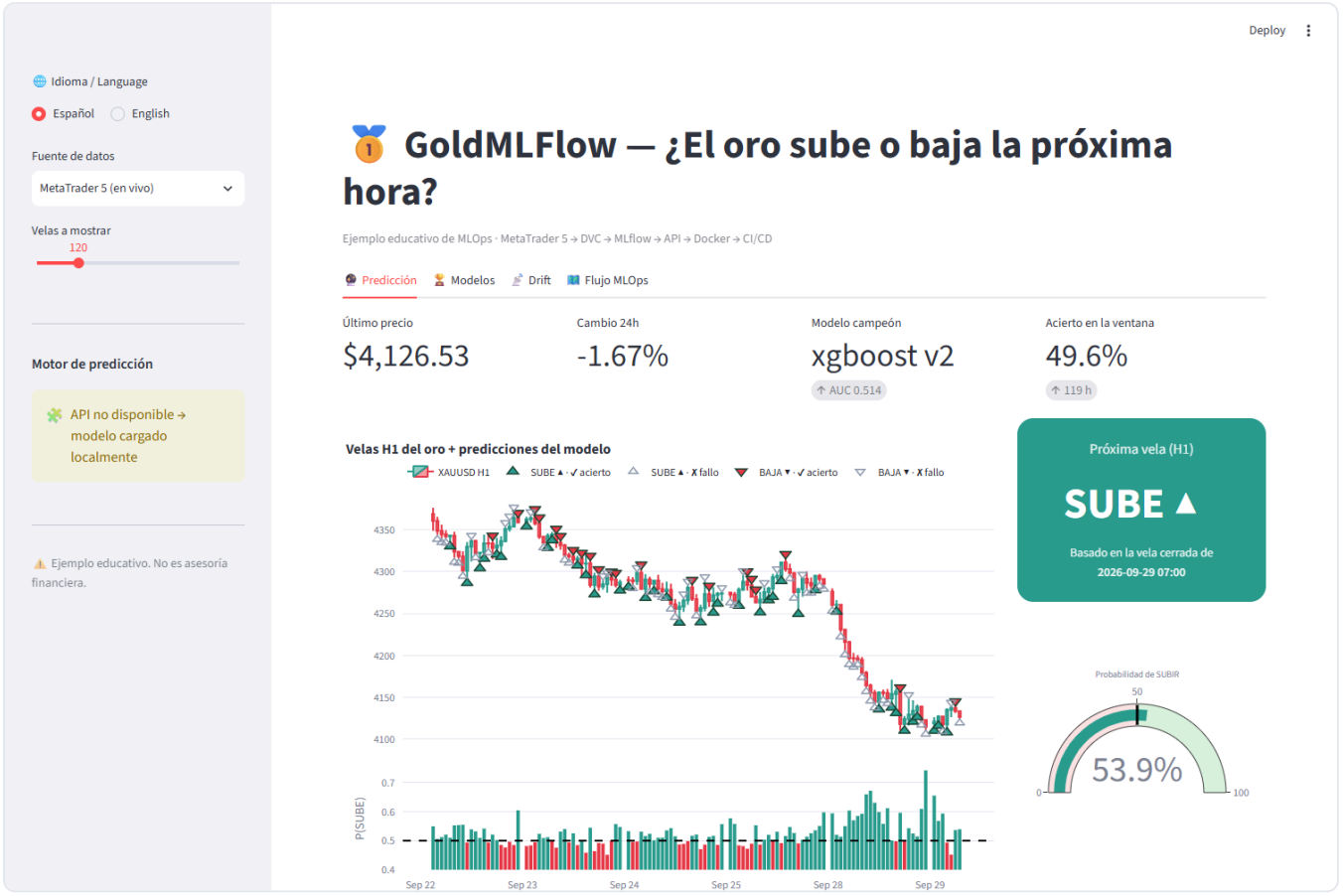
11 · Dashboard en Streamlit: ver y usar el modelo

🔍 **Analogía**	🪑 El comedor: donde los clientes (personas no técnicas) ven y disfrutan la comida
🎯 **Objetivo**	Mostrar todo el proyecto de forma visual a cualquier persona, en español o inglés
🏭 **Código**	`app/dashboard.py`
▶️ **Ejecutar**	`streamlit run app/dashboard.py` → http://localhost:8501 · o `docker compose up`

🧭 Barra lateral

Control	Opciones
🌐 Idioma	Español / English
Fuente de datos	**MetaTrader 5 (en vivo)** (solo Windows) · CSV de muestra en Git · subir tu propio CSV
Velas a mostrar	48 – 400
Motor de predicción	✅ API conectada → usa la API · si no, carga la misma app FastAPI dentro del proceso

🔮 Pestaña 1: Predicción



- **Tarjetas**: último precio, cambio en 24 h, modelo campeón + AUC y **acierto** del modelo en las velas visibles.
- **Tarjeta grande + indicador**: la predicción para la **próxima hora** (SUBE ▲ / BAJA ▼) y su probabilidad.
- **Gráfico de velas**: cada triángulo es lo que dijo el modelo *antes* de esa hora (▲ SUBE bajo la vela, ▼ BAJA encima). **Con color = acertó ✓, vacío gris = falló ✗.**
- **Barras de abajo**: P(SUBE) por hora; por encima de la línea punteada del 50 % → dijo SUBE.

🏆 Pestaña 2: Modelos

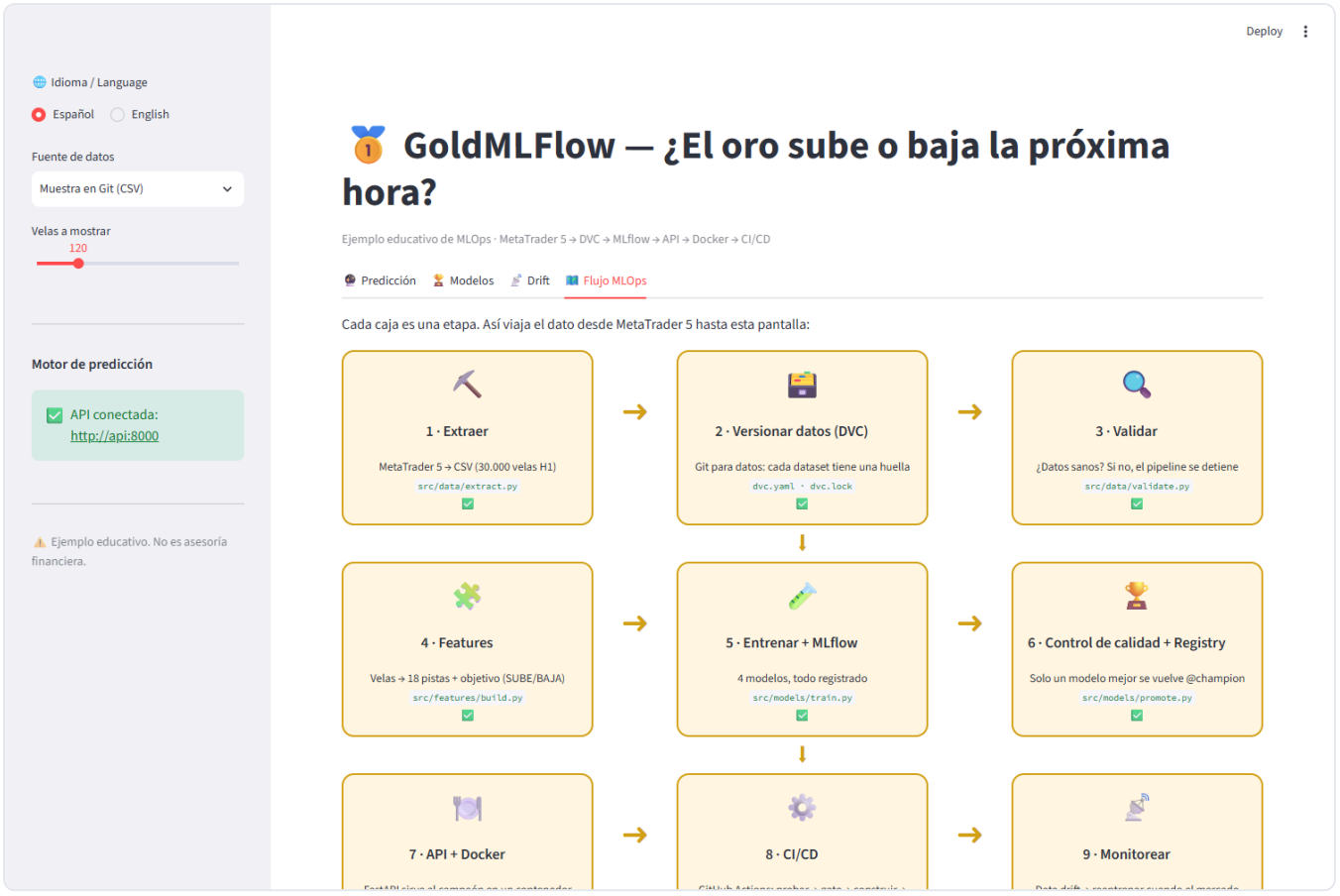


Tabla de posiciones contra la 🪙 moneda al aire, tabla de métricas, resultado del 🚦 control de calidad y backtest de juguete (modelo vs comprar y mantener).

📡 Pestaña 3: Drift

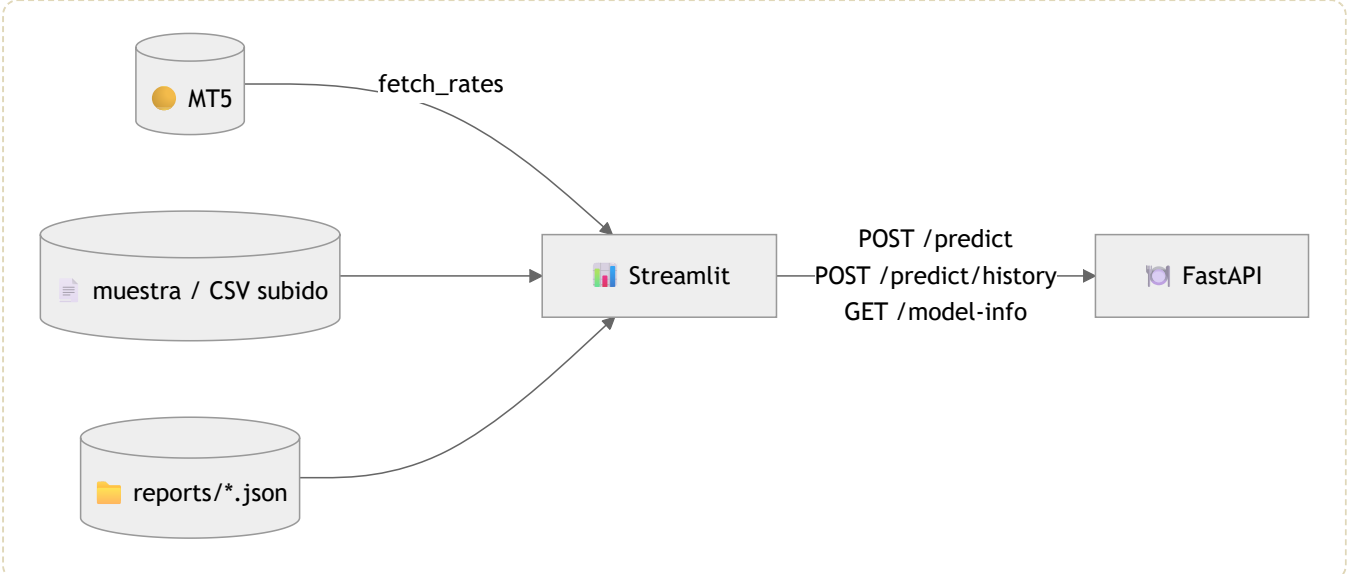
Qué pistas cambiaron más entre el periodo de entrenamiento y hoy, más la recomendación de reentrenar (ver capítulo 10).

🗺️ Pestaña 4: Flujo MLOps



Las 9 etapas como cajas: ✅ cuando existe la salida de esa etapa. Perfecta para **explicarle el proyecto a alguien que no sabe nada de MLOps**, caja por caja.

🧱 Cómo está construido



- El dashboard es un **cliente** de la API (separación de responsabilidades: interfaz ≠ modelo).
- Si la API no está corriendo, levanta la **misma** app FastAPI dentro del proceso (`TestClient`), así nunca duplica lógica.

- Revisa `/model-info`, no solo `/health`, para asegurarse de que lo que hay en ese puerto es *nuestra* API.

---

12 · Del notebook al código de producción

🍳 **Analogía**	🧪 El chef experimenta libremente en la cocina de pruebas (notebook); la receta ganadora se escribe en una ficha precisa que cualquier cocinero puede repetir (`.py`)
🎯 **Objetivo**	Entender **por qué y cómo** cada notebook se convirtió en un script de producción

🔀 Dos mundos, dos trabajos

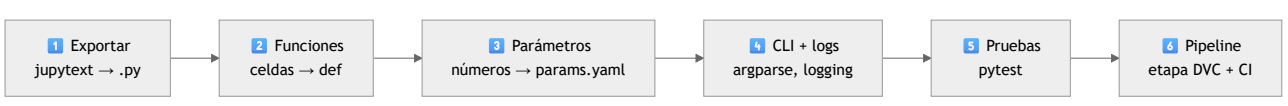
📓 Notebook	🚂 Script `.py`
Descubrir: probar, graficar, equivocarse	**Repetir**: el mismo resultado siempre
Las celdas corren en cualquier orden (¡estado oculto!)	Corre de arriba a abajo, siempre
Valores escritos a mano	Valores desde `params.yaml`
Tú miras el resultado	Logs, códigos de salida, reportes JSON
Difícil de probar / revisar en Git	Pruebas unitarias, code review, CI
Se ejecuta haciendo clic en ▶	Lo ejecutan DVC, el CI y Docker

🗺️ El mapa de este proyecto

Notebook	Se convirtió en	Prueba
`01_extract_mt5`	`src/data/extract.py`	— (necesita MT5)
`02_validate_explore`	`src/data/validate.py`	`tests/test_data.py`
`03_features`	`src/features/build.py`	`tests/test_features.py`
`04_train_mlflow`	`src/models/train.py`	pipeline de humo del CI
`05_evaluate_registry`	`src/models/promote.py` + `check_gate.py`	control de calidad del CI
`06_serve_api`	`src/api/main.py` + `Dockerfile`	`tests/test_api.py`
`07_monitor_drift`	`src/monitoring/drift.py`	pipeline de humo del CI

Cada notebook termina con una tabla "🔁 **Del notebook a producción**" que muestra exactamente qué cambió.

🪜 Los 6 pasos (hazlos en este orden)



1️⃣ Exportar el notebook a un `.py` plano

```
jupytext --to py:percent notebooks/03_features.ipynb -o notebooks/exported/03_features.py
```

Los resultados están en `notebooks/exported/`: el notebook como texto, con `# %%` marcando cada celda. **Es un punto de partida, no código de producción.**

2️⃣ Convertir celdas en funciones

```
# 📓 celda del notebook
X["rsi"] = 100 - 100 / (1 + gain / loss)
```

```
# 🏭 src/features/build.py
def rsi(close: pd.Series, window: int) -> pd.Series:
    """Relative Strength Index (0-100)."""
    ...
```

3️⃣ Mover los números mágicos a `params.yaml`

```
for w in [10, 50, 200]:                # 📓 escrito a mano
for w in p["sma_windows"]:             # 🏭 desde params.yaml → DVC re-ejecuta cuando cambia
```

4️⃣ Agregar línea de comandos y logs

```
parser.add_argument("--input", default=params["features"]["processed_path"])
log.info("%d rows x %d features | saved %s", ...)
```

→ el mismo script sirve para los datos completos (DVC) y para la muestra (CI).

5️⃣ Escribir pruebas para lo que revisaste "a ojo"

En el notebook 03 *vimos* la fuga de datos en un gráfico → en producción `test_no_leakage_features_ignore_the_future` lo verifica en cada push.

6️⃣ Conectarlo al pipeline

Agrega una etapa en `dvc.yaml` (deps, params, outs) y, si hace falta, un paso en `ci.yml`.

✅ Checklist: ¿mi script está listo para producción?

- [ ] Sin estado oculto: corre de arriba a abajo en una terminal limpia
- [ ] Sin rutas ni números escritos a mano → `params.yaml`
- [ ] `if __name__ == "__main__": main()`
- [ ] Errores claros y código de salida distinto de 0 si falla
- [ ] Al menos una prueba para su lógica crítica
- [ ] Declarado como etapa de DVC con deps/outs

- [ ] Pasa `ruff check`

---

13 · Glosario y preguntas frecuentes

📖 Glosario (en palabras simples)

Término	En palabras simples
MLOps	Prácticas para llevar modelos de ML de un notebook a producción de forma confiable y repetible
Vela (OHLCV)	Resumen de 1 hora de mercado: Apertura, Máximo, Mínimo, Cierre, Volumen
H1	Marco temporal de 1 hora por vela
Clasificación binaria	Predecir una de dos respuestas: SUBE (1) o BAJA (0)
Feature (pista)	Un número que se le da al modelo como pista
Target / etiqueta	La respuesta correcta que el modelo debe aprender
Baseline	El modelo razonable más tonto (siempre SUBE). Cualquier modelo real debe vencerlo
Fuga de datos (data leakage)	Una pista que esconde información del futuro → resultados falsamente buenos
Sobreajuste (overfitting)	Memorizar el pasado (ruido) en vez de aprender patrones
Validación walk-forward	Validación que siempre entrena con el pasado y prueba con lo que viene
ROC-AUC	Qué tan bien ordena el modelo las subidas por encima de las bajadas. 0,5 = moneda, 1,0 = perfecto
Pipeline (DVC)	Cadena de etapas con entradas y salidas declaradas; re-ejecuta solo lo que cambió
Remote / almacenamiento DVC	Donde viven las versiones de los datos (aquí una carpeta local; en empresas S3/GCS…)
Experiment tracking	Registrar parámetros, métricas y artefactos de cada entrenamiento (MLflow)
Run (ejecución)	Un intento de entrenamiento registrado en MLflow
Artefacto	Un archivo guardado con una ejecución (gráfico, modelo, reporte)
Firma del modelo (signature)	El "contrato" del modelo: qué entradas acepta y qué devuelve
Model Registry	Catálogo de versiones de modelos aprobados
Alias (@champion)	Una etiqueta movible que apunta a la versión en producción
Control de calidad (quality gate)	Reglas escritas que un modelo debe cumplir antes de ser promovido
API	Una puerta por la que los programas se comunican por HTTP
Endpoint	Una puerta específica de la API (p. ej. `/predict`)
Imagen / contenedor Docker	La caja empacada / la caja funcionando
Registry (ghcr.io)	Donde se guardan y comparten las imágenes Docker

Término	En palabras simples
CI	Revisiones automáticas en cada cambio
CD	Entrega automática de los cambios que pasaron el CI
Data drift	Los datos de hoy ya no se parecen a los de entrenamiento
Prueba KS	Prueba estadística que mide qué tan distintas son dos distribuciones (0–1)
Training/serving skew	Calcular las pistas distinto al entrenar y en producción (un error silencioso)

❓ Preguntas frecuentes

¿Me puedo hacer rico con este modelo? No. Acierta ~51,5 % y perdió contra comprar y mantener en el año de prueba. Es una herramienta para aprender. No es asesoría financiera.

`MT5 initialize() failed` Abre la terminal de MetaTrader 5 e inicia sesión (sirve una cuenta demo). El paquete de Python habla con la terminal abierta.

Mi broker no tiene `XAUUSD` Agrega su nombre en `params.yaml → data.symbol_candidates` (el script de extracción te muestra qué nombres encontró).

Uso Mac/Linux Todo funciona menos la extracción. Usa el CSV de muestra (ver 01 · Instalación).

`dvc repro` **dice "didn't change, skipping"** ¡Esa es la idea! Fuerza una etapa con `dvc repro -f <etapa>` (p. ej. `-f extract` para bajar datos nuevos).

`dvc pull` **falla en otro PC** El almacenamiento es una carpeta local de *tu* PC. Para equipos usa un remote compartido o en la nube: `dvc remote modify localstorage url s3://bucket/ruta`.

El puerto 8000 ya está en uso `$env:API_PORT=8010; docker compose up` (o `uvicorn src.api.main:app --port 8010`).

¿Por qué el CI entrena solo con 4.000 velas? El robot de GitHub no tiene MT5 ni acceso a tu almacenamiento DVC local. El CI verifica que el *código* funcione; el modelo *real* se entrena en tu PC y lo verifica el control de calidad.

¿Dónde veo los resultados del CI? https://github.com/MirandaCR/GoldMLFlow/actions → clic en una ejecución → **Summary**.

¿Cómo vuelvo a un modelo anterior? En la interfaz de MLflow (Models → gold-direction-h1) mueve el alias `@champion` a la versión anterior y luego vuelve a exportarlo con `python scripts/export_champion.py` y haz commit de `models/champion`.
